

Local LLM Question Log – Crash Course

Single-file end-to-end build of a FastAPI + Ollama + Postgres question log – v2

Swarup Biswas (SwarupSG)

v2 · 2026-07-08

Contents

Local LLM Question Log – A Crash Course	4
Why this course exists, and how it's different	4
How each module is structured	4
Prerequisites	5
What we're building	6
Choosing an IDE and an AI partner	6
Module 0 – Setup and Verification	7
The notional machine	7
The analogy	7
Install Python 3.11, 3.12, or 3.13	8
Clone or download this repo	8
Create a virtual environment	8
Install Postgres and create a superuser	9
Create the database and apply the schema	9
Install Ollama and pull the model	10
Trace what <code>verify_setup.sh</code> actually does	10
Predict before you run	11
Verify everything	11
Defend It	11
Module 1 – Hello FastAPI	11
The analogy	12
The notional machine	12
The cast (what each library does)	12
Install the first three packages	13
A quick aside: do you need to “restart the venv” after <code>pip install</code> ?	13
Write the app	13
Trace it in execution order	14
Predict before you run	15
Run it	15
Why this design	15
Verify	16
Defend It	16
Module 2 – POST and Pydantic Echo	16
The notional machine	16
The analogy	17
The cast (what's new this module)	17
Update <code>app/main.py</code>	17
Update <code>app/templates/index.html</code>	18
Trace it in execution order	19
Predict before you run	19
The three-error-codes drill	20

FastAPI just gave you a free debugger: <code>/docs</code>	20
Verify	21
Defend It	21
Module 3 – Call Ollama	21
The analogy	21
The notional machine	22
The cast (what’s new this module)	22
Add <code>httpx</code> to requirements	22
Update <code>app/main.py</code>	23
Trace it in execution order	24
Predict before you run	24
What’s in the JSON body to Ollama	24
About <code>try/except httpx.HTTPError</code>	25
Restart and try it	25
Verify	25
Defend It	25
Module 4 – System Prompt	26
The notional machine	26
The analogy	26
Update <code>app/main.py</code>	27
Trace it in execution order	27
Predict before you run	27
The before/after demo	28
Iteration exercise	28
Beyond the system prompt: <code>temperature</code> and other knobs	28
A note for AI-partner users	29
Verify	29
Defend It	29
Module 5 – Save to Postgres	30
The analogy	30
The notional machine	30
The cast (what’s new this module)	31
Add <code>psycpg</code> to requirements	31
Update <code>app/main.py</code>	31
Trace it in execution order	33
<code>/healthz</code> – a separate, lightweight version of the same probes	34
Why this design	34
Predict before you run	34
Try it	34
The persistence proof	35
Verify	35
Defend It	35
Module 6 – Read Recent History	35
The notional machine	35
The analogy	36
Update <code>app/main.py</code>	36
Why this design	38
Update <code>app/templates/index.html</code> to render the history	39
Trace it in execution order	40
Predict before you run	41
Try it	41
Verify	41
Defend It	41
Module 7 – Refactor into Layers	42

The notional machine	42
The analogy	42
The new file structure	42
app/main.py (now much shorter)	43
app/schemas.py (NEW)	44
app/database.py (NEW – and the long-awaited get_conn helper appears)	44
app/services/__init__.py (NEW – empty file)	45
app/services/ollama_service.py (NEW – SYSTEM_PROMPT lives here now)	45
app/services/interaction_service.py (NEW)	45
Trace it in execution order	46
Predict before you run	47
Try it	47
Verify	47
Defend It	47
Module 8 – Configuration via Environment	47
The notional machine	47
The analogy	48
The cast (what’s new this module)	48
Add python-dotenv to requirements	49
Create .env.example	49
Update app/main.py (load_dotenv at the very top, before service imports)	49
Update app/database.py	50
Update app/services/ollama_service.py	50
Why this design	51
The loud-fail demo	51
Trace it in execution order	51
Predict before you run	52
Verify	52
Defend It	52
What you’ve built	52
What comes next	53

Local LLM Question Log – A Crash Course

In this crash course, you will build the smallest **serious** AI web application from scratch – one fundamental at a time. By the end, you'll have a working app that takes a question in a browser, sends it to a local large language model running on your own machine, persists the answer in a database, and shows the most recent ten interactions. All running locally. No cloud accounts. No API keys.

The architecture in one line:

Browser → FastAPI → Ollama (llama3.2) → Postgres → Browser

This course is designed for adult learners with basic Python knowledge – you've written a function, you've used **pandas**, you've maybe touched a script that hits an API. You probably haven't run a long-lived web server, written real SQL, or coordinated three independent processes on one machine. You will, by Module 8.

The course is staged across **nine modules (0 through 8)**. Each module adds **exactly one** fundamental capability and ends with a working, runnable app. You don't need to finish all nine in one sitting – every module's checkpoint is a real app you can stop at, walk away from, come back to.

Why this course exists, and how it's different

There are a thousand "build a chat app" tutorials. This one is different in two ways:

- **One idea per module.** Module 1 is just "a web server is a long-running process." Not "a web server, plus routing, plus templates, plus Pydantic." Just the one thing. This gives you time to build a clean mental model of how every piece works before the next piece arrives.
- **Minimal, readable code.** You won't see frameworks-on-frameworks, ceremony, or features the running app doesn't need. If you've read tutorials and felt overwhelmed by setup that didn't seem to do anything, this one is the opposite of that.

Read on. By the end you'll have built – and be able to defend – a complete, locally-running, persistence-backed AI web application.

How each module is structured

Every module from 1 onward follows the same shape. This is deliberate – adult learners build understanding faster when the surface is predictable and the variation is in the *content*, not the format.

Section	What it does for you
The notional machine	A runnable mental model of <i>what the computer does</i> when this code runs. Most tutorials skip this and leave you to reverse-engineer the runtime from syntax. We don't.
The analogy	A concrete bridge from something you already know (a receptionist, a passport check, a kitchen) to the abstract concept the module introduces.
The code	The actual files. Full blocks, not diffs.
Trace it in execution order	A walkthrough of what happens <i>first, second, third</i> when the code runs – which is almost never the order the code is written in.
Predict before you run	A specific prediction prompt before you actually start the server. Surfacing your current mental model – and finding out where it's wrong – is the single most powerful learning move in this whole course.
Why this design	Why the code looks the way it does – what the alternatives were, what they would have cost, why the choice we made will be easier to maintain.
Run + verify	Concrete commands. The verify scripts (scripts/verify_module_N.sh) automate most of this.
Defend It	One question to sit with before you move on. Don't ask your AI partner to answer it. Reasoning through it yourself is the assessment.

Module 0 is slightly different because there's no application code yet – but the same underlying pattern (mental model, analogy, predict, verify) applies.

Prerequisites

You need:

- **Python 3.11, 3.12, or 3.13** (python3 --version should print one of these). 3.12 is the sweet spot. Python 3.14 may work but is bleeding-edge – if pip install fails for you on a package, downgrade to 3.12.
- **PostgreSQL 16 or newer** (16, 17, 18 all work – this guide installs 17 by default)
- **Ollama** with the llama3.2 model pulled (~2 GB download)
- About **10 GB of free disk space** (most of it for the Ollama model)
- A code editor (VS Code, Antigravity, Cursor, PyCharm – any modern editor)
- Comfortable on the command line: cd, running scripts, reading error messages
- ~2 hours per module for the first three modules; faster after that

On Windows, this whole stack runs inside WSL2 (Ubuntu). If you're on Windows and haven't installed WSL2 yet, do that first – see docs/module5-readiness-checklist.md §4 for the one-time wsl –install step, or docs/setup_walkthrough.md §1 for the hand-holding version. Every command in this crash course assumes a macOS Terminal **or** an Ubuntu (WSL2) terminal – the Linux commands work the same in both.

You do **not** need any cloud accounts, API keys, Docker, or knowledge of asyncio. The whole stack runs on your laptop.

What we're building

The final V1 app does this:

1. You open `http://localhost:8000` in your browser.
2. You see a textarea and an "Ask" button. Type a question. Click Ask.
3. Your browser sends the question to a FastAPI backend at `POST /ask`.
4. The backend wraps your question with a system prompt and sends it to a local LLM (llama3.2, running via Ollama on `localhost:11434`).
5. The LLM answers. The backend saves the question and answer to Postgres.
6. The backend returns the answer, plus the ten most recent interactions, as JSON.
7. Your browser shows the answer below the button and updates a "Recent" list.

The nine modules build this piece by piece:

Module	What gets added
0	Verified local environment (Python, venv, Postgres, Ollama, schema)
1	A web server that serves a static page
2	A typed POST endpoint that echoes the question
3	Replace the echo with a real LLM call
4	A system prompt shapes how the LLM responds
5	Persist every answer to Postgres
6	Show the recent ten interactions in the UI
7	Refactor the single file into routes / schemas / database / services
8	Move hardcoded values to environment variables

Each module's working app is tagged in the git repo (`v1-module-N-complete`) and has its own self-contained folder in `dist/module_NN_*/` if you want to skip ahead and run a particular checkpoint.

Choosing an IDE and an AI partner

This course was originally designed for **Google Antigravity + Gemini** – an agent-first IDE where the AI lives in a chat panel as a learning partner. The repo includes a `AGENTS.md` file that shapes how Gemini behaves in this project (it coaches rather than solving, refuses to jump ahead modules, and pushes back on doctrine-violating code).

You don't have to use Antigravity. The course works in any editor. If you're using VS Code with Copilot, Cursor with Claude, JetBrains, or even plain Vim – pick the one you have. The principles in `AGENTS.md` are written for Gemini but read as a perfectly good system prompt you can paste into Claude or ChatGPT to get similar coaching behavior from those tools.

What does matter:

- **Use an AI partner.** This course is designed to be built *with* an AI in the loop, not despite one. Each `dist/module_NN_*/README.md` includes a *"Try asking Gemini"* section with prompts you can paste into your AI of choice.

- **Don't let the AI write code you can't explain.** That's the test. If you copy code from your AI into your project and you can't tell a peer what each line does, slow down and ask the AI to explain it before you move on.
- **Don't ask the AI to answer the Defend-It questions.** Each module ends with one. Reasoning through it yourself is the whole point.

Now let's set up.

Module 0 – Setup and Verification

Single fundamental: every dependency your app will call must be reachable before any code runs.

This module has no application code. It exists because the most common reason a beginner's tutorial fails is *the environment isn't ready*, and they don't notice until line 47 of Module 3. We'll find out now.

Cohort students – read this first. If you've already completed pre-class setup via docs/module5-readiness-checklist.md or docs/setup_walkthrough.md, the install commands in the next several subsections (Python, clone, venv, Postgres, Ollama) duplicate what you already did. You don't need to redo them. What you DO want from this module:

- The **kitchen analogy** (below) – frames why setup is its own module
- The **"Trace what verify_setup.sh actually does"** section – the *why* behind each of the eight checks; this is the unique pedagogical content
- The **"Predict before you run"** prompt – surfaces gaps in your mental model
- Running ./scripts/verify_setup.sh – confirm everything is still working
- The **Defend It** question at the bottom

If verify_setup.sh surfaces a ✗ for any check, jump to the matching install subsection below to redo that one piece. **This module is written to stand alone for a self-paced learner reading only this crash course PDF; cohort students should treat the install commands as a reference, not a re-do.**

The notional machine

Your laptop is about to become the host for **three independent processes** that talk to each other over local network sockets:

1. **Python (your FastAPI app)** – the process that will eventually run when you type `uvicorn app.main:app`. Doesn't exist yet.
2. **Postgres** – the database server. A long-running background process. Listens on TCP port 5432.
3. **Ollama** – the LLM runtime. Another long-running background process. Listens on TCP port 11434.

When your FastAPI app eventually runs, it will open outbound connections to both Postgres (to save and read interactions) and Ollama (to get answers from the model). All three processes have to be alive *at the same moment*, on the same machine, listening on their expected ports, with credentials your app expects.

That's the model in your head: **three boxes, two arrows out from your app to the other two, three sockets open**. Module 0 makes sure all three boxes are running before you write any code.

The analogy

Setting up Module 0 is like **stocking and powering a kitchen** before your first day cooking professionally. The stove (Postgres) needs to be on. The fridge (Ollama with llama3.2) needs to be running and stocked with the ingredient (the 2 GB model file). The pantry (your venv with requirements.txt) needs to be in

a known state. The recipe book (the SQL schema in `sql/001_create_interactions.sql`) needs to be on the counter, opened.

If any of those is missing, you can't cook – and you'll discover the problem mid-recipe, with the heat already on. Better to discover it now, with nothing on the line.

Install Python 3.11, 3.12, or 3.13

If you already have any of these installed (`python3 --version` returns a 3.11.x / 3.12.x / 3.13.x line), skip this section. **3.12 is the sweet spot** – Ubuntu 24.04 ships it as `python3` by default.

```
# macOS (Homebrew)
brew install python@3.12

# Linux / WSL2 on Windows (Ubuntu 24.04 --- ships Python 3.12 by default)
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
```

Confirm:

```
python3 --version
# Expected: Python 3.12.x (or 3.11.x / 3.13.x if you already had one)
```

Ubuntu 24.04 + PEP 668 note. Trying `pip install` something outside a virtual environment on Ubuntu 24.04 errors with `externally-managed-environment`. That's intentional – Ubuntu wants you to use a `venv`. Which is exactly what the next section does.

Clone or download this repo

Where to put it. Clone into a sensible parent directory like `~/code/` or `~/dev/`. **WSL2 students: this must be inside your Ubuntu filesystem** (`~/code/` resolves to `/home/yourname/code/` inside Ubuntu) – never `/mnt/c/....`. Python operations on the `/mnt/c/` bridge are 10-50x slower because each file touch crosses the WSL/Windows boundary; `pip install` becomes painful and file-watching breaks. Inside Ubuntu's native filesystem, everything is fast.

```
mkdir -p ~/code && cd ~/code
git clone https://github.com/SwarupSG/fastapi-ollama-postgres-cohort.git
cd fastapi-ollama-postgres-cohort
```

(If you don't want to use `git`, click the green **Code** button on the GitHub page and "Download ZIP", then unzip and `cd` into the folder. WSL2 students: unzip into `~/code/` inside Ubuntu, not into a Windows folder.)

Create a virtual environment

A virtual environment isolates this project's Python packages from every other Python project on your machine. Without it, your `pip install fastapi==0.115.*` could conflict with another project's `fastapi==0.100.*`. Also: as the previous section flagged, Ubuntu 24.04 won't let you `pip install` outside a `venv` anyway.

Both platforms – same commands:

```
python3 -m venv venv
source venv/bin/activate
```

Your prompt should now start with `(venv)`. From now on, every terminal you open for this project needs to activate the `venv` first (just re-run `source venv/bin/activate` after `cd`-ing into the folder).

Install Postgres and create a superuser

macOS (Homebrew):

```
brew install postgresql@17
brew services start postgresql@17
```

Then create the postgres superuser the app expects. Homebrew creates only an OS-named role by default; the app's DATABASE_URL connects as the postgres user with password postgres, so we need that user:

```
psql -d postgres -c "CREATE USER postgres WITH PASSWORD 'postgres' SUPERUSER;"
```

If this says role "postgres" already exists, you're fine – move on.

Linux / WSL2 on Windows – use the official PostgreSQL apt repository (gives you version-pinned packages instead of whatever Ubuntu's archive points at). Three commands; run them one at a time:

```
# 1. Install the apt helper package (10-30 seconds):
sudo apt install -y postgresql-common

# 2. Run the official PostgreSQL apt setup script (interactive --- press Enter when
↳ prompted):
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh

# 3. Install PostgreSQL 17 (30-60 seconds):
sudo apt install -y postgresql-17
```

The second command prints a few lines and then asks Press Enter to continue or Ctrl-C to cancel. Press Enter. After it finishes you should see something like You can now start installing packages from apt.postgresql.org.

Start the service. Ubuntu 24.04 from wsl --install has systemd enabled by default, so:

```
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

The enable line tells systemd to auto-start Postgres when your Ubuntu session opens – so you don't have to manually start it next session.

Fallback: if systemctl errors with System has not been booted with systemd (rare on fresh installs but possible on older WSL2 instances), use `sudo service postgresql start` instead.

Set the postgres user's password to postgres (the value the app's DATABASE_URL expects):

```
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'postgres';"
```

Expected: ALTER ROLE.

Verify:

```
pg_isready -h localhost
# Expected: localhost:5432 - accepting connections
```

Create the database and apply the schema

```
createdb llm_question_log
psql -d llm_question_log -f sql/001_create_interactions.sql
```

You should see CREATE TABLE printed back.

Install Ollama and pull the model

macOS: download the .dmg from <https://ollama.com/download> (requires macOS 14 Sonoma or later), drag Ollama to Applications, open it once. The first launch installs the CLI tool and starts a background service.

```
# Then, from any Terminal:
ollama pull llama3.2
```

Linux / WSL2 on Windows – install with the official one-liner, then start the service:

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl start ollama
sudo systemctl enable ollama
ollama pull llama3.2
```

enable makes Ollama auto-start each time you open your Ubuntu terminal, so you don't have to start it manually next session. The ollama pull is the ~2 GB download – slowest step in the install. Let it finish before moving on.

Want faster inference using your GPU? WSL2-hosted Ollama is CPU-only – fine for the course, but inference takes ~5-20 seconds per response instead of ~3-7 seconds with GPU acceleration. If you have an NVIDIA / AMD GPU and want it, see docs/setup_walkthrough.md

Appendix – Optional: Run Ollama on the Windows host for GPU acceleration. The course code is identical either way.

Trace what verify_setup.sh actually does

Before you run the script, know what it's about to probe. Each  line is a separate check that maps to one part of the notional machine:

Check	What it actually does	What a failure means
Virtual environment active	Looks at \$VIRTUAL_ENV env var	You forgot to source venv/bin/activate in this terminal
Python 3.11.x	Runs python --version, parses output	venv was created with the wrong Python, or system Python is too old
requirements.txt present	[-f requirements.txt]	You're running the script from the wrong directory
Postgres reachable at localhost:5432	pg_isready -h localhost -p 5432	The Postgres process isn't running, or it's listening on a different port
Database llm_question_log reachable as user 'postgres'	psql "postgres://postgres:postgres@localhost:5432/llm_question_log" -c "SELECT 1"	The DB doesn't exist, or the postgres user doesn't exist, or has the wrong password
Table interactions exists	Queries information_schema.tables	You haven't applied the schema yet
Ollama reachable at localhost:11434	curl http://localhost:11434/api/tags	Ollama process not running, or wrong port
Model llama3.2 available	Greps the /api/tags JSON response for "name":"llama3.2"	Model not pulled – ollama pull llama3.2

Reading this table once means that when a check fails, you already know which of the three boxes in the notional machine has the problem.

Predict before you run

Before pressing Enter on the verify script, predict: **how many of the eight checks will pass on your first run?** Be honest with yourself. If you're following the steps in order, all eight should pass. If you skipped a step (e.g. you didn't `createdb`), some will fail.

If you predicted "all eight" and the script says otherwise, the gap between your prediction and reality is the most useful debugging signal you'll get all day. Read the failure message, fix it, re-run. Notice how much faster you converge when you predict first.

Verify everything

```
./scripts/verify_setup.sh           # macOS / Linux / WSL2
```

You should see eight green  lines ending with All checks passed. You're ready for Module 1.

If anything fails, the script tells you the one-line fix. Common failures:

- **Virtual environment not active** → run the `source venv/bin/activate` step above.
- **Postgres not reachable** → start the Postgres service. macOS: `brew services start postgresql@17`. Linux / WSL2: `sudo systemctl start postgresql`.
- **Cannot connect to llm_question_log as user 'postgres'** → run the `CREATE USER postgres` command above.
- **Database not found** → `createdb llm_question_log`.
- **Table 'interactions' not found** → run the schema apply step above.
- **Ollama not reachable** → start Ollama. macOS: open the Ollama app from Applications (or `brew services start ollama` if you installed via brew). Linux / WSL2: `sudo systemctl start ollama` (or `ollama serve` in a separate terminal).
- **Model llama3.2 not pulled** → `ollama pull llama3.2`.

Re-run the verify script after each fix. When it ends green, you're ready.

Defend It

Before moving on, sit with this question for a minute:

Why is "all dependencies reachable" worth a whole module before any application code is written?

You don't have to write down an answer. Just notice what comes to mind. Hold it as you build the next eight modules – you'll see the answer played out.

Module 1 – Hello FastAPI

Single fundamental: a web server is a long-running process that listens on a port and responds to HTTP requests.

That sentence sounds obvious. But if you've never built a web server before, the parts of it are not obvious at all. What does "long-running" mean? What is "a port"? What does the server *do* between requests? This module is about meeting those concepts in their simplest possible form: a server that returns a static page.

The analogy

Uvicorn is a **receptionist who never goes home**.

When the building opens (you start uvicorn), the receptionist sits down at the front desk and stays there. Every visitor (HTTP request) walks in; the receptionist looks at what they want (GET /), checks the directory (FastAPI's routing table) for the right person (the `index` function), passes the request along, takes back whatever that person hands over, gives it to the visitor, and turns to the next visitor in line.

The receptionist doesn't leave when the visitor leaves. The directory (`app`) doesn't get rebuilt for each visitor – it was built once when the building opened. **That's the "long-running" insight, and it's the entire reason a web server is different from a script.**

Hold that picture. Now let's see the actual mechanism behind it.

The notional machine

When you press Enter on `uvicorn app:main:app --reload`, **six** things happen – three when uvicorn starts, three for every request after.

At startup (these happen ONCE):

1. **Python loads `app/main.py` top to bottom.** The import lines pull in FastAPI's classes. `app = FastAPI(...)` creates one Python object in memory. `app.mount(...)` and `templates = Jinja2Templates(...)` configure that object.
2. **The `@app.get("/")` decorator registers `index` with `app`.** The function is **not called** – it's *registered*. After the file finishes loading, `app` knows: "if anyone ever asks me about GET /, run `index`." The function itself has not run yet.
3. **Uvicorn opens a TCP socket on port 8000 and blocks.** This is the "long-running" part. Without uvicorn, Python would finish executing the file and exit. Uvicorn keeps the process alive by sitting on the socket, listening for incoming connections.

For every request after (these happen on EVERY visitor):

4. **The browser asks for GET /.** The OS routes that to uvicorn, because uvicorn owns port 8000.
5. **Uvicorn hands the request to `app`.** It translates the raw HTTP bytes into a Python request object. `app` looks in its routing table, finds `index`, and calls `index(request)`.
6. **The return value goes back out.** Uvicorn takes whatever `index` returned, translates it back to HTTP bytes, and writes them to the socket. Browser sees the HTML. Cycle ends.

`app` stays in memory. Uvicorn keeps running. They're waiting for the next request.

That's the model. Hold it in your head while you read the code.

The cast (what each library does)

The five-line import block at the top of `app/main.py` brings in four pieces. Each has a specific, narrow job:

- **FastAPI** – the web framework. Provides the `FastAPI` class (the `app` object), the `@app.get(...)` decorator that registers routes, and the `Request` wrapper around incoming HTTP requests.
- **Uvicorn** – the ASGI server. Not imported in the code – you run it from the command line (`uvicorn app:main:app --reload`). Owns the TCP socket on port 8000 and feeds requests to `app`. **Uvicorn is the receptionist; FastAPI is her directory.** Two libraries, one analogy.
- **Jinja2Templates** – HTML template engine. Loads `app/templates/index.html` off disk, fills in any `{{ variables }}` you pass it, returns the rendered HTML as a string. In Module 1 the template has no variables – Jinja2 is just "loads the HTML file off disk." We'll use the variables for real in Module 6 when we render the recent-history list.

- **StaticFiles** – serves files (CSS, images, JS) **directly off disk** without going through a Python function. `app.mount("/static", StaticFiles(directory="app/static"), ...)` means: any URL starting with `/static/` → just hand back the matching file from `app/static/`. Faster and simpler than writing a route per file.
- **HTMLResponse** – a thin response class that tells FastAPI “*this body is HTML, set Content-Type: text/html.*” Without it, FastAPI defaults to JSON.

`app/main.py` is short because most of the heavy lifting (HTTP parsing, template rendering, static-file serving) is delegated to these pieces. You write the route; they handle the plumbing.

Install the first three packages

We’re going to start being disciplined about `requirements.txt`. Open it now and add these three lines (the file already has a comment header from Module 0; just append):

```
fastapi==0.115.*
uvicorn[standard]==0.32.*
jinja2==3.1.*
```

Then install:

```
pip install -r requirements.txt
```

Important: always edit `requirements.txt` first, then run `pip install -r requirements.txt`. Never just `pip install fastapi` and try to remember to update the file later – that’s how `requirements.txt` drifts out of sync with reality and the next person who clones your repo gets a broken setup.

A quick aside: do you need to “restart the venv” after pip install?

No. This trips up many beginners, so worth getting out of the way before we do it eight more times.

The venv is a *folder on disk* – `venv/` – containing its own python executable and its own `site-packages/` directory where `pip install` puts new packages. When you ran `source venv/bin/activate`, your shell edited its `PATH` so that typing `python` or `pip` resolves to the binaries inside that folder. Activation is a one-time-per-terminal change to your shell.

So the rules are:

- **After every pip install – no need to do anything to the venv itself.** The new packages are now sitting in `venv/lib/python3.12/site-packages/` (macOS, Linux, or WSL2 – the exact subdirectory depends on your Python minor version: 3.11, 3.12, 3.13). The next Python process you launch from this shell will see them on import. Done.
- **You DO need to restart uvicorn** so it re-imports the freshly-installed packages. The `--reload` flag watches your `.py` files for changes – it does NOT detect newly-installed packages. So whenever you `pip install` something, expect to `Ctrl+C` uvicorn and run `uvicorn app.main:app --reload` again.
- **In every NEW terminal you open**, run `source venv/bin/activate` again (after `cd`-ing into the project folder). Each terminal is a separate shell with its own `PATH`. The `(venv)` prefix in your prompt is your visual confirmation that you’re inside the venv.

That’s it. There’s no “restart the venv” step – the venv isn’t a running process. It’s just a folder of files; processes pick them up when they next import.

Write the app

Create the file structure:

```
app/
├─ main.py
```

```

├── templates/
│   └── index.html
└── static/
    └── style.css

```

app/main.py:

```

from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

```

That's the entire backend for Module 1. Five imports, one app, one mount, one templates object, one route.

app/templates/index.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Local LLM Question Log</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>
    <h1>Local LLM Question Log</h1>
    <p>Module 1: the server is running. Modules 2+ will add the form, the LLM, and the
    ↪ history.</p>
</body>
</html>

```

app/static/style.css:

```

body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
    max-width: 720px;
    margin: 2rem auto;
    padding: 0 1rem;
    line-height: 1.5;
}

```

Trace it in execution order

What happens *first*, *second*, *third* – which is almost never the order the code is written in.

At startup (when you run `uvicorn app.main:app --reload`):

1. `from fastapi import FastAPI, Request` → Python imports the FastAPI module into memory.

2. `from fastapi.responses import HTMLResponse` → and the response classes.
3. `from fastapi.templating import Jinja2Templates` → and the Jinja2 integration.
4. `from fastapi.staticfiles import StaticFiles` → and static file serving.
5. `app = FastAPI(title="...")` → **one** FastAPI object is constructed and assigned to the name `app`.
6. `app.mount("/static", ...)` → tells `app` “any URL starting with `/static/` should serve a file from `app/static/` on disk.” Now stored inside `app`.
7. `templates = Jinja2Templates(...)` → constructs a Jinja2 engine pointed at `app/templates/`. Stored in `templates`.
8. `@app.get("/", response_class=HTMLResponse)` followed by `def index(...)` → the decorator runs *now* (at import time), and it **registers** `index` as the handler for GET `/`. The function is *not* called.

After step 8, Python’s done loading the file. Uvicorn now opens TCP port 8000 and blocks, waiting for a connection.

On the first browser request (you visit `http://localhost:8000/`):

1. Browser sends GET `/` HTTP/1.1 over TCP to port 8000.
2. Uvicorn receives the bytes, parses them into a Python request object.
3. Uvicorn looks up GET `/` in `app`’s routing table → finds `index`.
4. Uvicorn calls `index(request=<the Request object>)`.
5. Inside `index`: `templates.TemplateResponse("index.html", {"request": request})` → Jinja2 reads `app/templates/index.html` from disk, processes it (no template variables to substitute in this case), wraps the result in an `HTMLResponse`.
6. `index` returns the `HTMLResponse`.
7. Uvicorn takes the response, writes the HTTP status line + headers + body bytes back to the TCP socket.
8. Browser receives the bytes, renders the page.

`index` exits. The `app` object stays in memory. Uvicorn goes back to waiting on the socket. **Steps 1-8 of “On the first browser request” repeat for every subsequent request.** Steps 1-8 of “At startup” never run again (unless you restart uvicorn).

Predict before you run

Before you actually start the server, decide:

- What HTTP status code will GET `/` return? (Easy.)
- What status will GET `/nonexistent` return? Why?
- What status will GET `/static/style.css` return? What’s `app` doing under the hood for that one?
- If you opened a second browser tab and hit GET `/` at the exact same moment, would there be one `app` object or two?

Now run uvicorn, hit each URL, and check whether your predictions held. Notice that you almost certainly *already had a model* in your head – even if you’ve never written a web server before. The model was *almost* right but probably not quite. That’s normal. That’s the model getting more precise.

Run it

```
uvicorn app.main:app --reload
```

Open **`http://localhost:8000`** in your browser. You should see the title and a paragraph.

Why this design

You might be wondering: *why no factory function? Why no router? Why no startup event?* Those are common patterns in production FastAPI. We don’t have them here because the lesson of Module 1 is

“a web server is a long-running process that responds to HTTP requests” – and adding routers and factories would make you read three more concepts before you understand the one we’re trying to teach. Production patterns earn their place when there’s enough code to justify them. There isn’t, yet.

Verify

Run the per-module verify script in another terminal (the server has to keep running):

```
./scripts/verify_module_1.sh
```

You should see ✓ GET / returns 200.

Defend It

Why `uvicorn app.main:app` and not `python app/main.py`?

Think it through. There’s a real reason. You’ll know you understand FastAPI when you can answer this without consulting docs.

Module 2 – POST and Pydantic Echo

Single fundamental: a request-response cycle has typed input and typed output. Validation lives at the boundary.

Now we’re going to add a form to the page and a `POST /ask` endpoint that *echoes* whatever you typed. We’re not calling an LLM yet – that’s Module 3. We’re learning the shape of a typed request-response cycle.

The notional machine

Module 1 had `app` and a single route. Module 2 keeps that, and adds three new things to the runtime:

1. **Two Pydantic classes (`AskRequest`, `AskResponse`).** These are also constructed at import time, just like `app` was. They’re not data – they’re *shape descriptions* that Pydantic uses to validate incoming JSON and serialize outgoing Python objects.
2. **A new route, `POST /ask`.** Registered into `app`’s routing table next to `GET /`. Same machinery as Module 1’s route.
3. **A two-stage validation pipeline that runs *before* your handler.** When a `POST /ask` arrives:
 - Uvicorn parses the HTTP request and hands it to `app`.
 - `app` looks at the route’s signature: `def ask(payload: AskRequest)`. The type hint `AskRequest` tells FastAPI: “*before calling this function, take the request body, parse it as JSON, validate it against the `AskRequest` schema, and only then construct an `AskRequest` object and pass it as `payload`.*”
 - **If validation fails, ask is never called.** FastAPI returns `422 Unprocessable Entity` with details about what was wrong, before your code sees the request at all.
 - If validation passes, your handler runs. It can now apply *business* rules (the `if not question` check) and raise `HTTPException(400, ...)` for value-level problems.

That’s the model: **two distinct gates, each at a different layer.** Pydantic guards the *shape* (is this a JSON object with a `question` field that’s a string?). Your handler guards the *value* (is the question, after stripping whitespace, actually non-empty?). They produce different status codes for the same reason – they’re different concerns.

The analogy

Module 2's validation pipeline works like **arriving at an international airport**:

- **Pydantic is the passport scanner at the gate.** It checks the *shape* of your document – is this even a valid passport? Right country? Required pages present? If the shape is wrong, you don't even meet the immigration officer; you get sent back with "your document is invalid" (a 422). The scanner doesn't care *why* you're traveling – that's not its job.
- **Your handler is the immigration officer.** Once Pydantic has confirmed your document is well-formed, the officer asks the substantive question: *what's the actual purpose? Is your stated purpose acceptable?* This is where the `if not` question check happens. Empty string after stripping = "your purpose for entry is blank" → rejected with a clear, business-level reason (a 400).

Two checks, two different concerns, two different rejection codes. **Don't try to make the passport scanner also check the purpose of travel** – that's the trap of putting `Field(min_length=1)` on `AskRequest`. You'd end up with both gates rejecting whitespace, in slightly different ways, and the system as a whole becomes incoherent.

The cast (what's new this module)

- **Pydantic** – runtime validation library, already installed as a FastAPI dependency. You declare a class like `class AskRequest(BaseModel): question: str` and Pydantic does two things: (a) parses the incoming JSON body into a Python object, (b) raises a 422 automatically if the body is malformed JSON, missing required fields, or has the wrong types. FastAPI uses Pydantic internally – when your handler signature says `payload: AskRequest`, FastAPI calls Pydantic to validate the request body **before** your function runs.

You're not adding a new server-side process this module – Pydantic is just a Python library inside your existing FastAPI process. The new thing is **typed input validation at the boundary**.

Update `app/main.py`

```
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

class AskRequest(BaseModel):
    question: str

class AskResponse(BaseModel):
    answer: str

@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

```
@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")
    return AskResponse(answer=f"You asked: {question}")
```

A note about the validation. You might be tempted to write `question: str = Field(min_length=1)` on `AskRequest`. Don't. Here's why: that would create two validation paths for the same idea – Pydantic rejecting "" with a 422, and the manual `if not question` rejecting whitespace-only input with a 400. The student (you) ends up wondering which is the truth.

Instead, we have **one** validation path: Pydantic checks the *shape* (is the field there, is it a string?) and the handler checks the *value* (is it empty after stripping?). Two distinct concerns, two distinct error codes:

- 422 (from Pydantic) means the request is malformed – wrong shape.
- 400 (from your handler) means the request is well-formed but the *value* is invalid.

This gives you one clear validation flow at the boundary, not two competing rules.

Update app/templates/index.html

Replace the body with:

```
<body>
  <h1>Local LLM Question Log</h1>
  <textarea id="question" rows="3" placeholder="Ask a question..."></textarea>
  <button id="ask-btn">Ask</button>
  <div id="answer"></div>
  <div id="error"></div>

  <script>
document.getElementById("ask-btn").addEventListener("click", async () => {
  const q = document.getElementById("question").value;
  const errEl = document.getElementById("error");
  const ansEl = document.getElementById("answer");
  errEl.textContent = "";
  ansEl.textContent = "Thinking...";
  try {
    const r = await fetch("/ask", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({question: q}),
    });
    const data = await r.json();
    if (!r.ok) throw new Error(data.detail || "Request failed");
    ansEl.textContent = data.answer;
  } catch (e) {
    ansEl.textContent = "";
    errEl.textContent = e.message;
  }
});
</script>
</body>
```

We're using vanilla `fetch()` – no React, no Vue, no jQuery. The frontend is a teaching surface, not a product.

Trace it in execution order

What happens when you click “Ask” with the textarea containing “hello”:

1. The click fires the JavaScript handler. `q = "hello"`.
2. JS calls `fetch("/ask", { method: "POST", ... body: JSON.stringify({question: "hello"}) })`. This issues a `POST /ask HTTP/1.1` request with body `{"question":"hello"}` and `Content-Type: application/json` to the same origin (your uvicorn).
3. Uvicorn receives the bytes, parses the HTTP request, hands it to `app`.
4. `app` matches `POST /ask` → finds the `ask` handler. Notices the parameter type hint `payload: AskRequest`.
5. **Pydantic gate:** FastAPI parses the JSON body into a Python dict, then validates it against `AskRequest`. There's a `question` field, it's a string. Valid. Pydantic constructs an `AskRequest(question="hello")` instance.
6. FastAPI calls `ask(payload=<that instance>)`.
7. Inside `ask`: `question = payload.question.strip() → "hello"`. The `if not question` check is `False`, so the handler doesn't raise.
8. Handler returns `AskResponse(answer="You asked: hello")`.
9. FastAPI sees `response_model=AskResponse` on the route, so it validates the return value against the schema (this is the *output* gate – same Pydantic, different direction).
10. FastAPI serializes the response to JSON: `{"answer":"You asked: hello"}`.
11. Uvicorn writes the HTTP response back to the socket.
12. Browser's `fetch` resolves with the response. JS reads `data.answer`, sets the answer div's text.

Now trace the **400 path** when the textarea is empty (`q = ""`):

1-4. Same as above, but body is `{"question":""}`. 5. Pydantic gate: `question` is present and is a string. Valid. Pydantic constructs `AskRequest(question="")`. 6. `ask` is called. 7. `question = "".strip() = ""`. `if not question` is `True`. Handler raises `HTTPException(400, ...)`. 8. FastAPI catches the exception, returns a 400 response with the detail message. **Steps 9-10 don't run.** Step 11 returns the 400.

Now trace the **422 path** when the body is `{}`:

1-4. Same as above, body is `{}`. 5. Pydantic gate: `question` field is missing. Validation fails. **The handler is never called.** FastAPI returns 422 with detail describing the missing field.

Notice: the 400 and 422 responses come from *different layers* of the system, even though they look similar to the browser. That's why they're different status codes.

Predict before you run

Before running the curl commands below, predict:

- What HTTP status will `{"question": " "}` (whitespace-only) return? Is the whitespace passing the Pydantic gate? Why or why not?
- What HTTP status will a body of `{"question": 42}` (an integer instead of a string) return? Which gate catches it?
- What about `{"question": "hello", "extra": "ignored"}` – does Pydantic care about extra fields by default?

Run each, see if you were right.

The three-error-codes drill

Your uvicorn ... --reload from earlier should already be running and will have auto-reloaded when you saved app/main.py. Watch the terminal for the WatchFiles detected change ... reloading line to confirm. If uvicorn isn't running for any reason, start it back up:

```
uvicorn app.main:app --reload
```

Then, in **another terminal**:

```
# 200 --- happy path
curl -s -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" -d '{"question":"hello"}'
# → {"answer":"You asked: hello"}

# 400 --- value error (handler's business rule)
curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" -d '{"question":""}'
# → {"detail":"Please enter a question."} HTTP 400

# 422 --- shape error (Pydantic, missing field)
curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" -d '{} '
# → {"detail":["...missing field..."]} HTTP 422
```

Now open the page in your browser, type something, click Ask. You should see “You asked: ...”. Try clicking Ask with the textarea empty – you should see the error message under the button.

FastAPI just gave you a free debugger: /docs

While uvicorn is still running, open **http://localhost:8000/docs** in your browser. You should see a page titled “Local LLM Question Log” with two endpoints listed: GET / and POST /ask.

Click POST /ask to expand it. Notice:

- The **request body** section shows a schema with one field – question: string, marked **required**. That came directly from class AskRequest(BaseModel): question: str. You wrote no documentation; FastAPI generated this from the type hint.
- The **responses** section shows two entries: **200** with the AskResponse schema (answer: string), and **422** with a generic validation error schema. The 422 is the same one you just hit with curl. Pydantic’s validator is what told FastAPI to advertise it.
- Scroll to the bottom – the **Schemas** section lists every Pydantic model in the app (AskRequest and AskResponse), each rendered as a typed object.

Now click “**Try it out**” on POST /ask. The schema view becomes editable. Replace the placeholder body with {"question": "hi"} and click **Execute**. You’ll see:

- The **curl** equivalent of the request (FastAPI generated this for you, ready to copy).
- The **response code**: 200.
- The **response body**: {"answer": "You asked: hi"}.

Try it again with body {} (empty object). Response code: 422. Response body: the Pydantic validation error.

Try it once more with {"question": " "} (whitespace only). Response code: 400. Response body: {"detail": "Please enter a question."}.

You just reproduced the entire three-error-codes drill without writing a single curl.

Why this matters beyond today: from Module 3 onwards, `/docs` is your fastest debugging tool. When something looks wrong in the browser, open `/docs`, click “Try it out”, paste the same payload your frontend was sending, and see exactly what the server returns – body, status, and headers all in one panel. No guessing about whether the bug is in the frontend or the backend.

There’s also `/redoc` (a read-only, magazine-style version of the same docs) – bookmark it if you prefer reading API surfaces over clicking buttons. Both come for free; both update automatically every time you add or change a route.

Verify

```
./scripts/verify_module_2.sh
```

Defend It

Why is 422 different from 400 here? Couldn’t they both be 400?

If you’ve never thought about HTTP status codes seriously, this is a good question to sit with. There’s a real distinction.

And while you’re there: open `/docs`. What did you write that produced that page? What did you *not* write?

Module 3 – Call Ollama

Single fundamental: a backend can call another local service over HTTP. Your app is a *client* of other services, not just a server to the browser.

So far, your FastAPI app has only been a *server* – something the browser calls. Now we’ll make it a *client* too – something that calls Ollama. This is the moment the architecture you’re building starts to look like real production AI apps. The model isn’t *in* your code; it’s a separate process that your code talks to over HTTP.

The analogy

You’ve just become **the wait staff at a restaurant**.

- **You (FastAPI/ask handler) are the waiter.** The customer (browser) sits at a table and tells you what they want – “I’d like the soup.”
- **The kitchen (Ollama) is a different team.** They have the actual cooking equipment and ingredients (the model weights). They don’t talk to the customer directly.
- **The waiter writes a ticket** (the JSON body to Ollama) and walks it to the kitchen window.
- **The kitchen produces the dish** (the LLM-generated answer) and hands it back through the window.
- **The waiter brings it to the table.**

A few things this analogy gets right that are easy to miss otherwise:

- **The kitchen and the waiter are independent processes.** If the kitchen is closed, you can take orders all day but you can’t fulfill any of them. (If Ollama isn’t running, your `/ask` returns 502.)
- **The waiter never goes into the kitchen.** They communicate through a defined interface (the order ticket / the HTTP request body). The kitchen could be replaced – same waiter, same tickets, food still arrives. (You could swap `llama3.2` for `mistral` by changing one constant.)
- **The customer doesn’t know the kitchen exists.** They just see “I asked, I got an answer.” (The browser has no idea Ollama is in the loop.)

Hold that picture. Now let's see the mechanism.

The notional machine

Up to Module 2, your machine had **two** processes that mattered: the browser, and your FastAPI process (uvicorn + app). The browser opened a TCP connection to uvicorn, sent an HTTP request, got a response. That was the whole picture.

In Module 3, a **third** process becomes load-bearing: **Ollama**. It's been running in the background since Module 0, but until now your code didn't talk to it. Now it does.

Walk through what happens during a single `POST /ask`:

1. **Inbound connection opens.** The browser opens a TCP connection to uvicorn on port 8000 and sends `POST /ask` with the question in the JSON body.
2. **Uvicorn hands the request to app.** Same as Module 2 – Pydantic validates the body, FastAPI calls `ask(payload)`.
3. **ask becomes a client.** Inside the handler, with `httpx.Client(timeout=60.0)` as `client`: constructs an HTTP client. Your code is now both a server (still talking to the browser) **and** a client (about to talk to Ollama).
4. **A second TCP connection opens.** `client.post(OLLAMA_URL, json=...)` opens an *outbound* TCP connection from your Python process to `localhost:11434` – Ollama's port. Your laptop now has **two TCP connections alive at once**: browser → uvicorn (inbound) and your handler → Ollama (outbound). Your FastAPI process is both ends of the network – server on one socket, client on another.
5. **Ollama processes the request.** Your handler is blocked, waiting. Ollama (the separate process) loads the model into RAM if it isn't already, runs inference, and writes the JSON response back over the outbound socket.
6. **The outbound connection closes.** When the `with httpx.Client(...)` block exits, your outbound connection to Ollama closes. The inbound connection from the browser is still open – you haven't responded yet.
7. **ask returns.** Your handler reads the answer text out of Ollama's JSON, returns an `AskResponse`. FastAPI serializes it; uvicorn writes it back over the inbound socket. The inbound connection closes.

app is back to waiting. Ollama is also back to waiting. The model stays loaded in Ollama's RAM, which is why the *next* request will be much faster than this one.

The cast (what's new this module)

- **httpx** – an HTTP client library for Python. We use it to make outbound HTTP requests *from* our FastAPI process to Ollama. The shape: `with httpx.Client(timeout=60.0) as client: r = client.post(url, json={...})`. The `with` block guarantees the client (and any open connection) is cleaned up even if an exception is raised mid-request.

Why `httpx` and not the more familiar `requests`? Two reasons: (a) `httpx` supports both sync and async with the same API, so we can graduate to async later without changing libraries; (b) `httpx` supports streaming responses cleanly, which we'll want when we eventually stream LLM tokens to the browser. For Module 3 we use the sync API only.

Add httpx to requirements

```
fastapi==0.115.*
uvicorn[standard]==0.32.*
```

```
jinja2==3.1.*
httpx==0.27.*
```

Then `pip install -r requirements.txt`.

We're using `httpx` instead of the more familiar `requests` because `httpx` supports both sync and async, and we want to leave the door open for streaming later. For Module 3 we use the sync API (`httpx.Client`).

Update `app/main.py`

```
import httpx
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

OLLAMA_URL = "http://localhost:11434/api/chat"
OLLAMA_MODEL = "llama3.2"

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

class AskRequest(BaseModel):
    question: str

class AskResponse(BaseModel):
    answer: str

@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")
    try:
        with httpx.Client(timeout=60.0) as client:
            r = client.post(OLLAMA_URL, json={
                "model": OLLAMA_MODEL,
                "messages": [{"role": "user", "content": question}],
                "stream": False,
            })
            r.raise_for_status()
            answer = r.json()["message"]["content"]
    except httpx.HTTPError:
```



```

    raise HTTPException(
        status_code=502,
        detail="Ollama is not reachable. Check that Ollama is running locally."
    )
    return AskResponse(answer=answer)

```

Trace it in execution order

When the browser POSTs {"question": "What is 2 + 2"}:

1. Inbound TCP: browser → uvicorn on port 8000.
2. Uvicorn parses the request, hands it to app.
3. app matches POST /ask → finds ask.
4. Pydantic validates the body against AskRequest → constructs AskRequest(question="What is 2 + 2").
5. ask is called. The if not question check passes.
6. with httpx.Client(timeout=60.0) as client: – a new httpx.Client is constructed. The with block guarantees it gets cleaned up.
7. client.post(OLLAMA_URL, json={...}) – httpx **opens a brand-new TCP connection out of your process** to localhost:11434, formats an HTTP POST /api/chat request with the JSON body, sends it, then *waits* (the function blocks) until Ollama responds or the 60-second timeout elapses.
8. Ollama (which has been idle, holding the model in RAM after the first call) processes the chat request. This takes anywhere from milliseconds (cached) to ~30 seconds (cold start of the model). It returns a JSON response shaped {"message": {"content": "...", ...}, ...}.
9. httpx receives the response. r.raise_for_status() checks for 4xx/5xx – none, so it doesn't raise.
10. r.json()["message"]["content"] digs into the JSON to pull out just the text answer.
11. The with block exits. **Outbound TCP connection closes.**
12. ask returns AskResponse(answer=<text>).
13. FastAPI serializes to {"answer": "4"} (or whatever the model said).
14. Uvicorn writes the response back over the *inbound* TCP connection.
15. Browser sees the answer, displays it.

If Ollama is **not running** when step 7 fires, the TCP connection refusal is raised by httpx as httpx.ConnectError, which is a subclass of httpx.HTTPError. The except block catches it and returns a 502 to the browser with a friendly message.

Predict before you run

Before testing this, predict:

- **First POST /ask after a fresh restart of Ollama** – how long does it take? 1 second? 10? 30? Why?
- **Second POST /ask with the same question** – does it take the same time, less, or more? Why?
- **What happens if you stop Ollama**, then ask a question – what status code? What's in the response body? Compare to the message you wrote in the except block. (To stop: macOS quit Ollama from the menu bar app (or brew services stop ollama if installed via brew); Linux / WSL2: sudo systemctl stop ollama.)
- **ps aux | grep ollama** before vs after stopping – what changes?

Try each. Most students discover that the first call is dramatically slower than subsequent ones because Ollama needs to load the model into RAM the first time. The trace above explains why: Ollama is a separate process with its own state, and that state isn't free to materialise.

What's in the JSON body to Ollama

The body has three keys:

- **model** – which model to use. Strings like `llama3.2`, `mistral`, `phi3`, etc. – whatever you’ve pulled with `ollama pull`. Run `ollama list` to see what’s available locally.
- **messages** – the conversation. A list of `{role, content}` dicts. Right now we only send one user message; in Module 4 we’ll add a system message in front of it.
- **stream** – whether the response comes back all at once (`false`) or token-by-token as the model generates it (`true`). We’re using `false` because it’s simpler – the response is one JSON blob and we read `.json()["message"]["content"]` directly. Streaming requires reading the response in chunks and is its own topic; we’ll touch it in a later module.

Ollama supports more keys (notably `options` for tuning sampling parameters like temperature) – we’ll meet them in Module 4. For now, three keys is enough.

About try/except `httpx.HTTPError`

The `try/except` covers two distinct failure modes with one narrow exception type:

- **Connection failures** – Ollama isn’t running, port 11434 isn’t open, network unplugged. `httpx` raises `httpx.ConnectError` (or similar), which is a subclass of `httpx.HTTPError`.
- **HTTP error status** – Ollama returns a 4xx or 5xx (e.g., model not found, bad request shape). `r.raise_for_status()` raises `httpx.HTTPStatusError`, also a subclass of `httpx.HTTPError`.

One catch handles both. Avoid the temptation to write `except Exception:` – that would also swallow programming bugs (typos, attribute errors, accidentally passing a string where an int was expected). You’d then silently report “Ollama unreachable” when the real problem is three lines above in your own code. Catching the narrowest type that expresses the failure mode you’re handling is one of the most under-rated habits in Python.

Restart and try it

```
uvicorn app.main:app --reload
```

In your browser, ask a real question – “What is the capital of France?”. The first call may take 20+ seconds because Ollama loads the model into RAM the first time. Subsequent calls are faster.

To prove Ollama is a separate process from your app, in another terminal:

```
ps aux | grep ollama | grep -v grep    # macOS / Linux / WSL2 --- all use the same shell
```

You’ll see the Ollama daemon running independently. Your FastAPI app cannot work without it being alive.

Verify

```
./scripts/verify_module_3.sh
```

Defend It

Why isn’t `llama3.2` imported as a Python package? Why does it have to be a separate process running outside the app?

This is a deeper question than it sounds. There are several real reasons. You’ll think of one or two; your AI partner can check whether you got the most important ones.

Module 4 – System Prompt

Single fundamental: an LLM call is a list of messages with **roles**. The system message shapes every response and is independent of the user's question.

This is the smallest code change in the whole course – one constant, one line in the messages array. It's also the most important conceptual jump. Before this module, you've been treating the LLM as a black box: you send text in, you get text out. After this module, you understand that the text you send is actually *a list of messages with roles*, and the system message is how production AI apps make models behave the way they want.

The notional machine

In Module 3, the body you sent to Ollama looked like this:

```
{"messages": [{"role": "user", "content": "What is 2+2?"}]}
```

You probably read that as “I’m sending a question.” That’s not quite what’s happening.

The truer model: **you are sending Ollama a conversation** – a list of turns, each with a role. In Module 3 that conversation has exactly one turn: a user said something. Ollama processes it, generates the next turn (an assistant message), and returns just the new text.

The messages list can hold an arbitrary number of turns with three legal roles:

- **system** – instructions that shape *all* of the model’s responses. Not seen by the user. Always processed first.
- **user** – what a human said.
- **assistant** – what the model said previously (used to build conversation history; we’ll touch this in a future module).

When Ollama processes a chat request, it reads the messages **in order**, builds an internal context that includes all of them, and generates the next response based on that context. **The system message at the top biases everything that comes after it.**

So the change in Module 4 – adding a system message before the user message – is not really a code change. It’s a change in *what conversation you’re sending the model into*. You’re handing it a one-line briefing before the user speaks.

The model is still doing the same thing it did in Module 3 (processing a list of messages and generating the next one). The list is just longer, and the first item shapes the rest.

The analogy

Think of **giving instructions to a temp who arrives at your office for one day**.

Without a system prompt, you point at the desk and say “*Here’s an inbox. Just respond to whatever comes in. Use your best judgment.*” They might write polite, careful, three-paragraph replies. They might write one-word replies. They might be funny, formal, technical, friendly – whatever their default is. You have no idea what you’ll get.

With a system prompt, before they even open the inbox, you hand them a card that says “*Today, you are responding as a concise, helpful assistant. Keep responses under 80 words. If a request would require info you don’t have, say so plainly instead of guessing.*”

Now they open the inbox. The first message says “*What is 2 + 2?*”. Their reply is short and direct, because that card is sitting on the desk in front of them, shaping how they read every incoming message.

The card never goes in the outbox. The customer never sees it. **The customer just sees a more consistent assistant.**

That card is the system message. The model reads it before every response. The user never sees it. That's it.

Update app/main.py

Add a `SYSTEM_PROMPT` constant near the other module-level constants:

```
OLLAMA_URL = "http://localhost:11434/api/chat"
OLLAMA_MODEL = "llama3.2"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)
```

And modify the messages array in the `/ask` handler:

```
"messages": [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": question},
],
```

That's the entire change. Two locations. About five new lines.

Trace it in execution order

When the browser POSTs `{"question": "What is 2 + 2?"}`:

1. (Same as Module 3, steps 1–6.) The request reaches `ask`.
2. `ask` builds the JSON body. The messages array now has **two** items: `json [{"role": "system", "content": "You are a concise, helpful assistant. Answer in one short paragraph (under 80 words). If you don't know, say so plainly."}, {"role": "user", "content": "What is 2 + 2?"}]`
3. `httpx` POSTs that to Ollama.
4. **Ollama reads both messages in order.** Internally, it builds a context window that includes the system instructions and the user question. The system message tells the model *how* to respond before the model has even fully read the question.
5. Ollama generates the next message in the conversation. The model's response is biased by what it just read at the top of the messages list – short, direct, willing to admit unknowns.
6. Ollama returns the response. (Same as Module 3 from here on.)

Notice what didn't change: the *protocol* between you and Ollama (POST `/api/chat` with `model`, `messages`, `stream`), the response shape (`{"message": {"content": "..."}})`, the way `httpx` works, the way FastAPI dispatches the route. Module 4 changed *what's in the messages list*, nothing else. That's why this is the smallest code change in the course but a huge conceptual jump.

Predict before you run

Before you reload the server and test, predict:

- For “What is 2 + 2?” – how many words will the answer be, with the system prompt active? Without it?
- For “What did I have for breakfast yesterday?” – what will the system-prompted version say? What will the un-prompted version say? (One of them will probably hedge for several sentences.)
- If you change the system prompt to “You always respond in the form of a question.”, will the model follow that? Always? Sometimes? Why not always?

Run each. Check your predictions. The gap between what you expected and what the model actually did is the lesson – *system prompts shape behavior, but they don't guarantee it. LLMs are non-deterministic.*

The before/after demo

You should see the LLM's behavior change before your eyes. Your `uvicorn ... --reload` from earlier modules is still running – it watches `app/main.py` for changes and auto-reloads when you save. So you don't need to manually restart uvicorn between the steps below; just save the file and ask again in the browser.

1. In your browser, ask: *"What is 2 + 2?"* – you should see a short, terse answer (one or two sentences). The system prompt is shaping it.
2. In `app/main.py`, **comment out** the system message line so the `messages` array is back to just `[{"role": "user", "content": question}]`. Save the file. Watch your terminal – uvicorn prints a `WatchFiles detected change ... reloading` line within a second.
3. Ask the same question again in the browser. You'll likely see a much longer, more elaborate answer – the model talks about decimal arithmetic, mathematical history, whatever.
4. **Restore** the system message line. Save. Ask again. Concise again.

Now try the same with a question the model can't actually know: *"What did I have for breakfast yesterday?"*. With the system prompt active, the model should admit it doesn't know in one sentence. Without it, expect a longer essay about how the model can't access personal information.

If uvicorn isn't running (you closed the terminal at some point, the auto-reload silently broke, etc.), the command to start it back up – from `dist/module_04_system_prompt/` with your `venv` active – is:

```
uvicorn app.main:app --reload
```

The `Ctrl+C` to stop, and the same line to restart.

You've just discovered the single most important technique for shaping production AI behavior. That's it. That's the whole module.

Iteration exercise

Before moving on, write your own system prompt. Try one that: - Forces answers in haiku format - Always cites a date in the answer - Constrains the model to one-word answers

Predict what you think will happen *before* you run each one. Then run it and compare. The model will follow some prompts more reliably than others – that's a real and important lesson about how non-deterministic LLMs are.

Beyond the system prompt: temperature and other knobs

The system prompt is the most important way to shape an LLM's output, but it's not the only one. Ollama (and almost every LLM API) supports an `options` block in the request body for controlling how the model picks its next word. The most-used setting is `temperature`. You don't need it for this course, but you'll bump into it everywhere else, so it's worth understanding.

temperature controls how predictable vs creative the model is. It's a single number, typically between 0.0 and 1.0:

- **temperature: 0.0** – fully deterministic. The model always picks the most-likely next word. Same question → same answer, every time. Best for: factual lookups, code generation, structured output, anything where you'd rather have a boring right answer than a surprising one.

- **temperature: 0.2** – mostly deterministic with a tiny bit of variation. Same question gives almost-identical answers, with occasional minor wording changes. Good default for assistants, summarization, classification.
- **temperature: 0.7** – a fair amount of creative variation. Same question can give noticeably different answers each time. Good for: brainstorming, casual conversation, creative writing, generating multiple options.
- **temperature: 1.0** – maximum variation that’s still coherent. Each answer can take a quite different path. Good for: poetry, fiction, “give me a wild idea” prompts.
- **Above 1.0** (some APIs go up to 2.0) – the model starts making increasingly weird choices. Words become unusual; sentences may drift off-topic. Mostly only useful if you specifically want surprising output.

If you wanted to try this in our app, you’d add `"options": {"temperature": 0.2}` to the JSON body sent to Ollama. Ask “What is the capital of France?” five times. Then change to `0.9` and ask the same five times. Notice the variation.

A few other knobs that show up in Ollama requests, in plain language:

- **top_p** (also called nucleus sampling, default ~0.9) – caps how rare a word the model is allowed to pick. Lower = more focused; higher = more varied. Usually safe to leave alone and just adjust temperature.
- **format: "json"** – forces the model to return strictly-valid JSON. Useful if you’re building something that programmatically parses the output. Without this, the model often returns prose like *“Sure! Here’s the JSON: {...}”* which breaks `json.loads()`.
- **num_predict** – maximum number of tokens to generate. A “token” is roughly a word piece – 100 tokens is roughly 75 words. Useful for capping cost on hosted models or capping latency.

You don’t need any of these for our course. But once you know they exist, you’ll recognize them in every LLM API you touch – OpenAI, Anthropic, Google, AWS Bedrock, Cohere – they all use the same vocabulary.

A note for AI-partner users

If you’re using Claude, ChatGPT, Cursor, or Antigravity as your AI partner, what you just built is exactly how those tools work too. There is a system prompt – written by Anthropic / OpenAI / Google – that shapes every conversation you have with them. You can usually find a published version on their developer documentation pages. Read one. Notice the shape. The thing you just learned to write for `llama3.2` is the production version of what’s been guiding every chat you’ve ever had with an AI assistant.

If you’re using Antigravity specifically, this repo has a file called `AGENTS.md` – that’s the system prompt for the Gemini sessions in this project. Open it. It’s the actual production thing.

Verify

```
./scripts/verify_module_4.sh
```

This is a soft check (the script can verify `SYSTEM_PROMPT` exists in code and the endpoint responds, but it can’t strictly enforce “the model followed the under-80-words rule” because LLMs are non-deterministic). Inspect the answer verbosity yourself.

Defend It

Why does the system prompt go into the same `messages` array as the user’s question? Why isn’t it a separate API parameter?

This question has more depth than it appears. Sit with it.

Module 5 – Save to Postgres

Single fundamental: an application persists state in a database. The state outlives the request.

Up to now, every interaction is forgotten as soon as the response goes out the door. That's fine for a demo but not for a real app. Module 5 makes every answer *persist* – you'll be able to stop the server, restart it, and the data is still there.

The analogy

Think of your Python process as **your short-term memory** and Postgres as **a paper notebook**.

When you read a phone number off a screen and dial it five seconds later, you're holding it in short-term memory. Useful, instant, free – but if someone interrupts you for thirty seconds, it's gone. The phone number didn't survive the interruption.

When you write the same phone number in a paper notebook, the act of writing is slower. It costs ink. It costs a page. But six months later, you can flip back to that page and the number is still there. The notebook *survived your nap, your weekend, your forgetting*.

A Python list is short-term memory. Postgres is the notebook.

In Module 5, every time your handler gets an LLM answer, it does two things: 1. Hands the answer back to the browser (short-term memory: useful right now). 2. Writes it down in the notebook (Postgres: useful later, including after you restart the app).

The handler does both in sequence. The user sees one answer; the system has two copies of it briefly, then one durable copy after the response is sent.

Hold that picture. Now let's see the mechanism.

The notional machine

The big idea in Module 5 is **a boundary crossing**: data leaves your Python process for a place where it survives the process dying. Two pictures together explain how – the connection picture and the state picture.

The connection picture (extends Module 3).

In Module 3 you added one outbound TCP connection: your FastAPI process talking to Ollama on port 11434. In Module 5 you add a **second** outbound connection: your FastAPI process talking to Postgres on port 5432. During a single `POST /ask`, your laptop has **three** TCP connections live at the peak:

- **Inbound:** browser → uvicorn (port 8000)
- **Outbound to Ollama:** your handler → Ollama (port 11434)
- **Outbound to Postgres:** your handler → Postgres (port 5432)

The state picture (the genuinely new idea).

In Modules 1–4, every piece of state your app cared about lived **inside the Python process** – local variables, request/response objects, the app object itself. When the Python process exited, that state vanished. Restart uvicorn? Everything is gone.

In Module 5, the data crosses a boundary. Walk through what your handler does after Ollama replies:

1. **Build the row in Python.** A tuple of (question, answer, model_name).
2. **Open a connection to Postgres.** `psycopg.connect(DATABASE_URL)` opens a TCP socket to port 5432.

3. **Run the INSERT.** The row leaves Python's memory and travels over the socket. Postgres receives it and writes it to disk.
4. **Call `conn.commit()`.** This tells Postgres "this is final, durable, never lose it." Without commit, the write is held in a transaction that disappears if the connection drops.
5. **Close the connection.** The row is now on Postgres's disk, **independent of your Python process.**

Try it yourself:

```
# In your uvicorn terminal: Ctrl+C to stop the Python process.

# Restart it:
uvicorn app.main:app --reload

# In another terminal, query the table:
psql "postgresql://postgres:postgres@localhost:5432/llm_question_log" \
-c "SELECT id, LEFT(question, 40), created_at FROM interactions ORDER BY id DESC LIMIT 1;"
```

The row is still there – because it isn't *in* Python anymore; it's on Postgres's disk. That's persistence.

Internalize the distinction: state in your Python process is **ephemeral** (lives until the process dies); state in Postgres is **durable** (lives until you delete it).

The cast (what's new this module)

- **psycopg** (specifically `psycopg[binary]`, version 3) – the Postgres driver for Python. Lets you open a connection to a running Postgres process, send SQL, and read results back. Two patterns matter:
 - with `psycopg.connect(DATABASE_URL)` as `conn`: opens a connection; the connection closes automatically when the block exits.
 - with `conn.cursor()` as `cur`: opens a cursor (Postgres's query handle) inside that connection; same auto-close behavior.

We use `[binary]` (rather than plain `psycopg`) so pip downloads precompiled wheels for your platform (macOS, Linux, WSL2 – all covered) instead of trying to compile native code locally. This is the difference between "it just works" and "it fails because you don't have a C compiler installed."

Postgres itself is the separate process that's been running on your machine since Module 0. `psycopg` is just the *client library* that talks to it over a TCP socket – same shape as `httpx` talking to Ollama, just a different wire protocol underneath.

Add psycopg to requirements

```
fastapi==0.115.*
uvicorn[standard]==0.32.*
jinja2==3.1.*
httpx==0.27.*
psycopg[binary]==3.2.*
```

Then `pip install -r requirements.txt`. We're using `psycopg[binary]` because the binary distribution avoids native build steps – pip downloads pre-compiled wheels for your platform.

Update app/main.py

```
import httpx
import psycopg
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
```



```

from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

OLLAMA_URL = "http://localhost:11434/api/chat"
OLLAMA_MODEL = "llama3.2"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)
DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/llm_question_log"

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

class AskRequest(BaseModel):
    question: str

class AskResponse(BaseModel):
    answer: str

@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")

    try:
        with httpx.Client(timeout=60.0) as client:
            r = client.post(OLLAMA_URL, json={
                "model": OLLAMA_MODEL,
                "messages": [
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": question},
                ],
                "stream": False,
            })
            r.raise_for_status()
            answer = r.json()["message"]["content"]
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Ollama is not reachable.")

    try:

```

```

        with psycopg.connect(DATABASE_URL) as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO interactions (question, answer, model_name) "
                    "VALUES (%s, %s, %s)",
                    (question, answer, OLLAMA_MODEL),
                )
            conn.commit()
    except psycopg.Error:
        raise HTTPException(
            status_code=502,
            detail="Postgres is not reachable. Check your database connection."
        )

    return AskResponse(answer=answer)

@app.get("/healthz")
def healthz():
    status = {"ollama": False, "postgres": False}
    try:
        with httpx.Client(timeout=5.0) as client:
            client.get("http://localhost:11434/api/tags").raise_for_status()
            status["ollama"] = True
    except httpx.HTTPError:
        pass
    try:
        with psycopg.connect(DATABASE_URL) as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
            status["postgres"] = True
    except psycopg.Error:
        pass
    return status

```

Trace it in execution order

When the browser POSTs {"question": "What is the capital of France?"}:

1. (Same as Module 4, steps 1–6.) The request reaches ask. Pydantic validates. The handler calls Ollama, gets back the answer text.
2. **NEW:** with psycopg.connect(DATABASE_URL) as conn: – psycopg parses the URL, opens a TCP socket to Postgres on port 5432, performs the authentication handshake (sends postgres username and password), and gets back a connection handle. The with block ensures the connection closes cleanly even if something raises inside.
3. with conn.cursor() as cur: – gets a cursor object that can run queries on this connection.
4. cur.execute("INSERT INTO interactions ... VALUES (%s, %s, %s)", (question, answer, OLLAMA_MODEL)) – psycopg substitutes the parameters safely (no SQL injection – the %s is parameter binding, not string formatting), serializes the SQL + values, sends them over the TCP socket to Postgres.
5. **Postgres receives the INSERT, checks the schema, allocates a new row id (auto-incremented from SERIAL PRIMARY KEY), writes the row to its in-memory page cache.** At this point, the data exists in Postgres's RAM but **not yet** durably on disk.
6. conn.commit() – psycopg sends a COMMIT to Postgres. Postgres flushes the relevant pages to its

write-ahead log (WAL), calls `fsync()` on the WAL file, and only *then* sends back an acknowledgement. After step 6 returns, **the data is durable: it would survive a power loss right now.**

7. The `with conn.cursor()` block exits, releasing the cursor.
8. The `with psycopg.connect()` block exits, closing the TCP connection to Postgres.
9. Handler returns the answer to the browser. The user sees the response.

If you stop uvicorn (Ctrl+C) right now, the Python process exits. The `app` object, all module-level constants, the cursor and connection – all gone. **The Postgres row remains** because step 6 happened.

That `commit` step is what makes the difference between “I think I saved it” and “it’s saved.”

/healthz – a separate, lightweight version of the same probes

The `/healthz` endpoint runs the same kinds of operations (TCP to Ollama, TCP to Postgres) with much shorter timeouts and queries that do nothing useful (`SELECT 1` is the standard “is the database alive” probe). It returns a small JSON object that an operator (or a load balancer) can poll to know whether the app’s dependencies are reachable.

Why this design

You might be wondering why we don’t extract `with psycopg.connect(DATABASE_URL) as conn:` into a helper function. With only two call sites today (`/ask` writes, `/healthz` reads), a helper would save almost nothing – one duplicated line – while adding a layer a future reader has to trace through to understand the code. We’ll add the helper in Module 7, when there are three places that need it and the savings start to outweigh the cost.

This is a small but real production engineering instinct: **don’t create abstractions ahead of evidence that you need them.** The cost of waiting is one duplicated line; the cost of building the wrong abstraction is months of unwinding it.

The `/healthz` route catches `httpx.HTTPError` and `psycopg.Error` *specifically*, not bare `Exception`. Bare `except Exception:` would also swallow programming bugs (`NameError`, `TypeError`, `AttributeError`) and silently report “Postgres unreachable” when the real cause is a typo three lines up. Always catch the narrowest exception that expresses the failure mode you actually intend to handle – this single habit saves more debugging time than almost anything else you can do.

Predict before you run

Before you run the curls below, predict:

- Ask a question. Then immediately stop uvicorn and restart it. Re-run the `SELECT`. **Predict:** is the row still there? Why?
- Now stop uvicorn AND stop Postgres. Restart only uvicorn. Hit `/healthz`. **Predict the JSON response.** Then restart Postgres and check again. (To stop Postgres: macOS `brew services stop postgresql@17`; Linux / WSL2 `sudo systemctl stop postgresql`.)
- What happens if you forget the `conn.commit()`? (Try it: comment out the line, ask a question, query the table from a separate `psql` session. Does the row appear?)
- If the Ollama call succeeds but the Postgres `INSERT` fails (e.g. Postgres goes down between steps 6 and 7 of the trace), what does the user see? Is the LLM’s answer “lost”? Should it be?

These are the questions that distinguish someone who *uses* a database from someone who *thinks* about it.

Try it

```
uvicorn app.main:app --reload
```

```
# Both services up
curl -s http://localhost:8000/healthz
# → {"ollama":true,"postgres":true}

# Ask a question
curl -s -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question":"What is the capital of France?"}'

# Confirm the row landed in the DB
psql "postgresql://postgres:postgres@localhost:5432/llm_question_log" \
  -c "SELECT id, LEFT(question,40), LEFT(answer,60), created_at FROM interactions ORDER BY
  ↵ id DESC LIMIT 1;"
```

The persistence proof

Now do this:

1. Stop uvicorn with Ctrl+C in the terminal where it's running.
2. Restart it with `uvicorn app.main:app --reload`.
3. Re-run the SELECT above.

The row is still there. *That* is the difference between a Python list and a database.

Verify

```
./scripts/verify_module_5.sh
```

Defend It

Why save to a database instead of an in-memory Python list? What does the database give us that a list does not?

There's an obvious answer (persistence) and several non-obvious ones. Try to find at least three.

Module 6 – Read Recent History

Single fundamental: an application reads state back from the database to inform what the user sees.

Saving without reading is half a feature. Module 6 shows the most recent ten interactions on the page, both on initial load and after every new question. We'll add a `/history` endpoint that returns the same data as JSON, and the `/ask` response will start including the recent history alongside the answer.

The notional machine

In Module 5 you opened a TCP socket to Postgres to **write** data (INSERT). In Module 6 you open the same kind of socket to **read** data (SELECT). The runtime mechanics are nearly identical:

1. Open a TCP connection to Postgres.
2. Send SQL over it (this time a SELECT, not an INSERT).
3. Postgres reads the relevant rows from disk (or its page cache), assembles a result set, and streams it back over the same socket as a sequence of rows.

4. psycopg parses the rows; with `row_factory=dict_row`, each row becomes a Python dict(`{"id": 1, "question": "...", "answer": "..."}`) instead of a positional tuple.
5. Your code wraps each dict in a Pydantic Interaction model (which validates the shape) and returns the list.
6. FastAPI sees `response_model=list[Interaction]`, serializes the list to JSON, sends it back to the browser.

A second new piece: **the /ask response now contains both the new answer AND the full recent history**. The handler does the LLM call, does the INSERT, then does a SELECT to fetch the freshly-updated history, and returns all of it as one JSON object. The browser uses both pieces – the new answer goes into the answer div, the history goes into the Recent list. *One round trip from browser, two database operations server-side.*

The browser also separately calls GET /history on initial page load, so the Recent list appears even before the user asks anything.

The analogy

If Module 5 was *writing in the notebook*, Module 6 is **flipping back through the notebook**.

Same notebook. Same pages. Different direction – instead of adding a new entry, you're scanning the most recent ten and reading them. The `ORDER BY id DESC LIMIT 10` is "show me the most recent ten in reverse chronological order" – which is exactly how you'd read a notebook to see what just happened.

The notebook itself is unchanged by reading from it. (That's the difference between a SELECT and an INSERT – selects don't mutate.) But the *act of reading* is what makes the previously-saved data useful. Without it, you've been writing into a notebook nobody ever opens.

Update app/main.py

```
import httpx
import psycopg
from psycopg.rows import dict_row
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

OLLAMA_URL = "http://localhost:11434/api/chat"
OLLAMA_MODEL = "llama3.2"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)
DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/llm_question_log"

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

class AskRequest(BaseModel):
    question: str
```

```

class Interaction(BaseModel):
    id: int
    question: str
    answer: str
    model_name: str
    created_at: str


class AskResponse(BaseModel):
    answer: str
    history: list[Interaction]


def fetch_recent_history(limit: int = 10) -> list[Interaction]:
    with psycopg.connect(DATABASE_URL) as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                "SELECT id, question, answer, model_name, "
                "       to_char(created_at, 'YYYY-MM-DD HH24:MI:SS') AS created_at "
                "FROM interactions ORDER BY id DESC LIMIT %s",
                (limit,),
            )
            return [Interaction(**row) for row in cur.fetchall()]


@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})


@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")

    # ... Ollama call (unchanged from Module 5) ...
    try:
        with httpx.Client(timeout=60.0) as client:
            r = client.post(OLLAMA_URL, json={
                "model": OLLAMA_MODEL,
                "messages": [
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": question},
                ],
                "stream": False,
            })
            r.raise_for_status()
            answer = r.json()["message"]["content"]
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Ollama is not reachable.")

```

```

# ... Postgres INSERT (unchanged from Module 5) ...
try:
    with psycopg.connect(DATABASE_URL) as conn:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO interactions (question, answer, model_name) "
                "VALUES (%s, %s, %s)",
                (question, answer, OLLAMA_MODEL),
            )
        conn.commit()
except psycopg.Error:
    raise HTTPException(
        status_code=502,
        detail="Postgres is not reachable. Check your database connection."
    )

return AskResponse(answer=answer, history=fetch_recent_history())

@app.get("/history", response_model=list[Interaction])
def history():
    return fetch_recent_history()

@app.get("/healthz")
def healthz():
    # ... unchanged from Module 5 ...
    status = {"ollama": False, "postgres": False}
    try:
        with httpx.Client(timeout=5.0) as client:
            client.get("http://localhost:11434/api/tags").raise_for_status()
            status["ollama"] = True
    except httpx.HTTPError:
        pass
    try:
        with psycopg.connect(DATABASE_URL) as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
            status["postgres"] = True
    except psycopg.Error:
        pass
    return status

```

Why this design

Two design choices worth understanding:

created_at: str plus SQL to_char(...) instead of datetime plus Pydantic JSON encoder. Why? Because the database is great at formatting dates – that’s literally what `to_char` is built for. Pushing the formatting into Pydantic with a custom JSON encoder would add three lines of Pydantic config to avoid one line of SQL, and the SQL version reads better. A useful general instinct: if a layer of your stack is already good at something, let it do that thing instead of replicating it elsewhere.

fetch_recent_history is the first extracted helper in this app. It’s called from two places (`/ask` and `/history`), and the alternative is duplicating an eight-line SQL block in both. The function name

(`fetch_recent_history`) also makes the intent clearer than re-reading the SQL twice and parsing what it's doing. When extracting a helper saves you from copy-pasting non-trivial logic AND gives the operation a name that explains itself, it's worth doing – even with only two callers.

Update `app/templates/index.html` to render the history

Replace the body with:

```
<body>
  <h1>Local LLM Question Log</h1>
  <textarea id="question" rows="3" placeholder="Ask a question..."></textarea>
  <button id="ask-btn">Ask</button>
  <div id="answer"></div>
  <div id="error"></div>

  <h2>Recent</h2>
  <ol id="history">No questions yet.</ol>

  <script>
    function renderHistory(items) {
      const list = document.getElementById("history");
      if (!items || items.length === 0) {
        list.innerHTML = "<li>No questions yet.</li>";
        return;
      }
      list.innerHTML = items.map(it =>
        `<li>
          <div class="q">${escapeHtml(it.question)}</div>
          <div class="a">${escapeHtml(it.answer)}</div>
          <div class="meta">${it.model_name} · ${it.created_at}</div>
        </li>`
      ).join("");
    }

    function escapeHtml(s) {
      return s.replace(/([<>"']/g, c => ({
        "&": "&amp;", "<": "&lt;", ">": "&gt;", "\"": "&quot;", "'": "&#39;";
      }[c]));
    }

    async function loadHistory() {
      const r = await fetch("/history");
      if (r.ok) renderHistory(await r.json());
    }

    document.getElementById("ask-btn").addEventListener("click", async () => {
      const q = document.getElementById("question").value;
      const errEl = document.getElementById("error");
      const ansEl = document.getElementById("answer");
      errEl.textContent = "";
      ansEl.textContent = "Thinking...";
      try {
        const r = await fetch("/ask", {
          method: "POST",
          headers: {"Content-Type": "application/json"},
        });
      }
    });
  </script>

```



```

        body: JSON.stringify({question: q}),
    });
    const data = await r.json();
    if (!r.ok) throw new Error(data.detail || "Request failed");
    ansEl.textContent = data.answer;
    renderHistory(data.history);
  } catch (e) {
    ansEl.textContent = "";
    errEl.textContent = e.message;
  }
});

loadHistory();
</script>
</body>

```

And add a few CSS rules to `app/static/style.css`:

```

h2 {
  margin-top: 2rem;
  font-size: 1.1rem;
}

#history {
  padding-left: 1.2rem;
}

#history li {
  margin-bottom: 1rem;
}

#history .q {
  font-weight: 600;
}

#history .a {
  white-space: pre-wrap;
}

#history .meta {
  font-size: 0.85rem;
  color: #666;
}

```

Trace it in execution order

On initial page load (browser opens `http://localhost:8000/`):

1. Browser GETs `/`. `index` returns `index.html`.
2. Browser parses the HTML, sees the `<script>` block, runs it.
3. `loadHistory()` fires immediately (it's called at the bottom of the script). It does `fetch("/history")`.
4. That triggers a GET `/history` to your server.
5. Handler `history()` runs → calls `fetch_recent_history()`.
6. `fetch_recent_history` opens a Postgres connection, runs the SELECT, gets back rows as dicts, wraps each in `Interaction`, returns the list.

7. FastAPI serializes the list of Interaction objects to JSON.
8. Browser gets the JSON, calls `renderHistory(items)`, which builds an HTML list and replaces the contents of `<ol id="history">`.

On asking a new question (user clicks Ask):

1-8. (Same as Modules 4 and 5.) Browser POSTs to `/ask`. Handler validates, calls Ollama, does the INSERT. 9. **NEW:** Handler calls `fetch_recent_history()` *after* the INSERT. The freshly-inserted row is included in the result because the INSERT's commit already happened. 10. Handler returns `AskResponse(answer=..., history=[...10 most recent...])`. 11. FastAPI serializes both fields. Browser receives `{"answer": "...", "history": [...]}`. 12. JS sets the answer div, then calls `renderHistory(data.history)` to update the Recent list.

Notice: **the user clicks once, the server hits Postgres twice** (once to write, once to read back the most recent 10). That's intentional – the read is cheap, and bundling the response means the browser doesn't have to make a second round-trip.

Predict before you run

Before testing:

- On the very first page load (no rows in the DB yet), what does the Recent list show? Where does the “No questions yet” text come from – server or browser?
- Ask 12 questions in a row. How many are in the Recent list now? Why exactly that number? Where in the code is the limit set?
- Run `curl http://localhost:8000/history` from a separate terminal *while* uvicorn is busy answering an `/ask` (which can take 5–30 seconds). Predict: does `/history` block waiting for `/ask`, or does it respond immediately? Why?
- Try editing the SQL query to remove `ORDER BY id DESC`. What does Postgres return? Is it deterministic? (Hint: no, and that's a real database lesson.)

Try it

Save your changes. If uvicorn is still running with `--reload` from earlier, it auto-reloads – watch the terminal for the `WatchFiles detected change ... reloading` line. Refresh the browser. The “Recent” section appears. Ask a question – when the answer comes back, the Recent list updates immediately with your new question at the top.

If uvicorn isn't running, start it: `uvicorn app.main:app --reload`.

```
curl -s http://localhost:8000/history | python3 -m json.tool
```

Verify

```
./scripts/verify_module_6.sh
```

Defend It

Why does `/ask` return the history bundled with the answer instead of letting the browser call `/history` separately?

Both designs work. There's a trade-off here that's worth thinking through.

Module 7 – Refactor into Layers

Single fundamental: a maintainable codebase separates concerns. Each file has one reason to change.

By the end of Module 6, your `app/main.py` is doing a lot: routes, schemas, database connection, Ollama call, business logic. That's been fine through Module 6 because there wasn't enough code to make splitting worth the navigation cost. Now there is – and the database-connection logic in particular has reached three call sites, which is enough that extracting it into one place pays off.

No user-visible change in Module 7. The acceptance test is that every Module 6 verification still passes byte-for-byte. We're moving code, not changing what it does.

The notional machine

This is the most counter-intuitive module of the course. **The runtime call graph is identical to Module 6.** When a `POST /ask` arrives:

- Same TCP connections open in the same order.
- Same HTTP requests fly between the browser, FastAPI, Ollama, and Postgres.
- Same SQL gets executed.
- Same JSON comes back.

What's actually different is the **import graph**. In Module 6, Python loads one file (`app/main.py`) and everything is defined there. In Module 7, Python loads `app/main.py`, sees the `from app.database import get_conn`, opens `app/database.py`, runs it, then continues. Same for `app.schemas`, `app.services.ollama_service`, `app.services.interaction_service`. Each module's top-level code runs *once*, in import order, and after they've all loaded, the runtime call graph collapses back to exactly what Module 6 had.

So the module count goes from 1 to 5, but at runtime there's no extra cost. The benefit is at *human-reading* time: when you want to find “where does the Ollama call live?”, you go straight to `services/ollama_service.py` instead of scrolling through a 100-line `main.py`.

There's one subtle runtime change worth noting: **the new `get_conn()` context manager**. Instead of with `psycopg.connect(DATABASE_URL) as conn:` written out at three call sites, there's now with `get_conn()` as `conn:`. Functionally equivalent – `get_conn` is a generator wrapped with `@contextmanager` that opens a connection, yields it, and closes it. But it's now defined in one place and used in three. (See “Why this design” below for why three is the magic number.)

The analogy

Think of the Module 6 codebase as a **kitchen with everything piled on the counter**. The salt, the knives, the recipes, the pans, the spices – all reachable, all in the same workspace, but mixed together. You can find the salt by scanning the counter. The first time you cook a meal, that's actually fine. By the fifteenth meal, you've spent a cumulative half-hour scanning for things.

The Module 7 refactor is **putting things in labeled drawers**. Same salt. Same knives. Same recipes. The food you cook tomorrow tastes identical to the food you cooked yesterday. But now: knives in the knife drawer, spices in the spice rack, recipes in a binder on the shelf. When you want a knife, you go to the knife drawer.

The kitchen *does the same thing*. The cook just spends less time looking for things. **That's maintainability** – you didn't gain a feature, you reduced the cost of every future change.

The new file structure

```
app/
├─ main.py           # FastAPI app, routes only
├─ schemas.py        # AskRequest, AskResponse, Interaction
```

```

└─ database.py          # DATABASE_URL, get_conn (introduced HERE, not earlier)
└─ services/
    └─ __init__.py      # empty
    └─ ollama_service.py # OLLAMA_URL, OLLAMA_MODEL, SYSTEM_PROMPT, call_ollama()
    └─ interaction_service.py # save_interaction, fetch_recent_history

```

SYSTEM_PROMPT migrates from app/main.py (where it lived since Module 4) into services/ollama_service.py – the constant lives with the function that uses it.

app/main.py (now much shorter)

```

import httpx
import psycpg
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles

from app.database import get_conn
from app.schemas import AskRequest, AskResponse, Interaction
from app.services.ollama_service import call_ollama
from app.services.interaction_service import save_interaction, fetch_recent_history

app = FastAPI(title="Local LLM Question Log")

app.mount("/static", StaticFiles(directory="app/static"), name="static")
templates = Jinja2Templates(directory="app/templates")

@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")
    answer = call_ollama(question)
    save_interaction(question, answer)
    return AskResponse(answer=answer, history=fetch_recent_history())

@app.get("/history", response_model=list[Interaction])
def history():
    return fetch_recent_history()

@app.get("/healthz")
def healthz():
    status = {"ollama": False, "postgres": False}
    try:
        with httpx.Client(timeout=5.0) as client:
            client.get("http://localhost:11434/api/tags").raise_for_status()

```

```

        status["ollama"] = True
    except httpx.HTTPError:
        pass
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
            status["postgres"] = True
    except psycopg.Error:
        pass
    return status

```

app/schemas.py (NEW)

```

from pydantic import BaseModel

class AskRequest(BaseModel):
    question: str

class Interaction(BaseModel):
    id: int
    question: str
    answer: str
    model_name: str
    created_at: str

class AskResponse(BaseModel):
    answer: str
    history: list[Interaction]

```

app/database.py (NEW – and the long-awaited get_conn helper appears)

```

import psycopg
from contextlib import contextmanager

DATABASE_URL = "postgresql://postgres:postgres@localhost:5432/llm_question_log"

@contextmanager
def get_conn():
    with psycopg.connect(DATABASE_URL) as conn:
        yield conn

```

This is where the get_conn helper appears. Three places now need a database connection: `save_interaction`, `fetch_recent_history`, and `/healthz`. Without `get_conn`, you'd write the same `with psycopg.connect(DATABASE_URL) as conn:` line three times. With it, the connection logic lives in one place – and if you ever need to change it (add timeouts, add pooling, switch to a different driver), you change one file instead of three. That's what the helper is buying you.

app/services/__init__.py (NEW – empty file)

This is here so Python treats services/ as a package.

app/services/ollama_service.py (NEW – SYSTEM_PROMPT lives here now)

```
import httpx
from fastapi import HTTPException

OLLAMA_URL = "http://localhost:11434/api/chat"
OLLAMA_MODEL = "llama3.2"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)

def call_ollama(question: str) -> str:
    try:
        with httpx.Client(timeout=60.0) as client:
            r = client.post(OLLAMA_URL, json={
                "model": OLLAMA_MODEL,
                "messages": [
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": question},
                ],
                "stream": False,
            })
            r.raise_for_status()
            return r.json()["message"]["content"]
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Ollama is not reachable.")
```

app/services/interaction_service.py (NEW)

```
import psycpg
from psycpg.rows import dict_row
from fastapi import HTTPException

from app.database import get_conn
from app.schemas import Interaction
from app.services.ollama_service import OLLAMA_MODEL

def save_interaction(question: str, answer: str) -> None:
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO interactions (question, answer, model_name) "
```

```

        "VALUES (%s, %s, %s)",
        (question, answer, OLLAMA_MODEL),
    )
    conn.commit()
except psycopg.Error:
    raise HTTPException(
        status_code=502,
        detail="Postgres is not reachable. Check your database connection."
    )

def fetch_recent_history(limit: int = 10) -> list[Interaction]:
    with get_conn() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                "SELECT id, question, answer, model_name, "
                "       to_char(created_at, 'YYYY-MM-DD HH24:MI:SS') AS created_at "
                "FROM interactions ORDER BY id DESC LIMIT %s",
                (limit,),
            )
            return [Interaction(**row) for row in cur.fetchall()]

```

Trace it in execution order

At startup, Python's import resolver walks the dependency graph:

1. uvicorn imports `app.main`.
2. `app/main.py` line 1: `import httpx` → loaded (or already cached if imported before).
3. `app/main.py` line 2: `import psycopg` → loaded.
4. `app/main.py`'s `from fastapi import ...` lines.
5. **from `app.database` import `get_conn`** → Python opens `app/database.py`, runs it top-to-bottom: `import psycopg`, `from contextlib import contextmanager`, `DATABASE_URL = "..."`, the `@contextmanager` decorator runs and wraps `get_conn`. Returns to `main.py` with `get_conn` available.
6. **from `app.schemas` import `AskRequest`, `AskResponse`, `Interaction`** → opens `app/schemas.py`, runs it (defines the three Pydantic classes), returns.
7. **from `app.services.ollama_service` import `call_ollama`** → opens `app/services/__init__.py` (empty), then `app/services/ollama_service.py`, runs it (defines `OLLAMA_URL`, `OLLAMA_MODEL`, `SYSTEM_PROMPT`, `call_ollama`), returns.
8. **from `app.services.interaction_service` import `save_interaction`, `fetch_recent_history`** → opens that file. **Note the chain:** `interaction_service.py` itself does `from app.database import get_conn` (already loaded – Python returns the cached module), `from app.schemas import Interaction` (cached), `from app.services.ollama_service import OLLAMA_MODEL` (cached). Then defines its two functions. Returns.
9. Back in `main.py`: `app = FastAPI(...)`, the mount, the templates, the four route registrations.
10. Uvicorn opens port 8000, starts listening.

On a POST `/ask`: the call sequence is `ask` → `call_ollama(question)` → `save_interaction(question, answer)` → `fetch_recent_history()`. Each helper does what its respective code did in Module 6 – exact same Postgres SQL, exact same Ollama HTTP call, exact same JSON shape. **The trace through the runtime is byte-for-byte equivalent to Module 6's trace.** What changed is *where in source code* each step lives.

Predict before you run

Before testing:

- Run **every Module 6 verify command** (`/healthz`, `POST /ask` with three error code variants, `GET /history`). Predict every response. Then run them. Are any of the responses different from Module 6? They shouldn't be, by even a single byte.
- `git diff --stat` between Module 6's tag and Module 7's tag, scoped to `app/`. Predict what files appear in the diff. Are any *behavior* changes? (Hint: look for changes to function bodies, not just file moves.)
- The `get_conn` helper is now used in three places. Imagine you had introduced it back in Module 5, when there were only two callers. What would you have gained? What would you have *lost* (think about future readers and the cost of jumping between files to follow a simple line of code)? There's no objectively right answer to "when is the right time to extract a helper" – but reasoning through this trade-off concretely is one of the most useful instincts a working developer can build.

Try it

`index.html` and `style.css` are unchanged from Module 6. With `uvicorn` running (`uvicorn app.main:app --reload` if it isn't already), refresh the browser. **Everything should work exactly the same.** Ask a question. Watch the history update. Hit `/healthz`. All identical to Module 6.

That's the point. The user sees nothing new. But the codebase is now organised so that, if you wanted to swap from Ollama to a hosted LLM, you'd touch one file: `services/ollama_service.py`. If you wanted to add a second persistence target (say, also write to a log file), you'd touch one file: `services/interaction_service.py`. That's maintainability.

Verify

```
./scripts/verify_module_7.sh # run from inside dist/module_07_refactor_layers/ if testing
↪ in dist
```

The script confirms behavior unchanged AND that the file split happened.

Defend It

We didn't change any behaviour. What did we gain?

This is the question that separates engineers from typists.

Module 8 – Configuration via Environment

Single fundamental: code describes behaviour. Configuration describes environment. The same code runs against a different database by changing one environment variable.

This is the V1 final state. After this module, the same `app/` folder runs against your local Postgres on your laptop, against a staging Postgres on a remote server, against a production Postgres on a cloud – by changing one file (`.env`), without touching any code.

The notional machine

Every Python process has an attribute called `os.environ` – a dict-like object that contains all the **environment variables** the operating system passed to that process when it started. You can read from it (`os.environ["DATABASE_URL"]`), write to it (rare), and you can list everything in it (`os.environ.keys()`).

When you run `uvicorn app.main:app --reload`, your Python process inherits whatever environment variables your shell already had set. By default, that's stuff like `PATH`, `HOME`, `USER` – things your laptop set for you. *Your* app's variables (`DATABASE_URL`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`) are not in that environment unless you put them there.

`python-dotenv` adds a small step: when `load_dotenv()` runs, it reads `.env` from the current directory, parses it as `KEY=VALUE` lines, and **adds each line to `os.environ`** for the current Python process. After that, `os.environ["DATABASE_URL"]` succeeds.

The critical sequencing detail: **`load_dotenv()` must run BEFORE any module that reads `os.environ` at import time.** Look at the new `app/main.py` – `load_dotenv()` is the very first thing, before any other import. That's because the moment you from `app.database import get_conn`, Python executes the top of `database.py`, which says `DATABASE_URL = os.environ["DATABASE_URL"]`. If `load_dotenv()` hasn't run yet, that line raises `KeyError` and **import itself fails** – the server never starts.

That `KeyError` at import time is the entire safety mechanism. If `.env` is missing, the app refuses to start with a clear, instant, source-pinpointed error. **No silent defaults. No “wait, why is it pointing at the wrong database?” three weeks into production.**

The analogy

Think of an environment variable as **a setting on a thermostat**.

The HVAC system (your app's logic) is the same in winter and summer. It blows air through ducts. It senses temperature. It activates heating coils or AC compressors. *That* doesn't change.

What changes between dev/staging/prod is the *thermostat setting*. In your house: 68°F in winter, 74°F in summer. The HVAC works exactly the same; the temperature it targets is configuration. In your app: `DATABASE_URL=localhost` on your laptop, `DATABASE_URL=staging-db.internal:5432` in staging, `DATABASE_URL=prod-cluster.aws.com:5432` in production. Same code. Different setting.

Now imagine the thermostat is broken – there's no setting at all. What should the HVAC do? Two philosophies:

- **“Be helpful, pick a default.”** The HVAC guesses 70°F. Sometimes you'll be cold; sometimes you'll be hot; sometimes the default's fine. You won't notice the broken thermostat until summer when you keep wondering why the house feels off.
- **“Refuse to start until configured.”** The HVAC blinks an error light and won't run. You notice immediately. You fix the thermostat. Now everything works correctly.

We use the second approach: `os.environ["X"]` (which raises `KeyError` if missing) instead of `os.environ.get("X", default)` (which silently returns the default). **Crash loudly so the operator sees the misconfiguration before it bites a user.** This is one of the most production-relevant habits you'll build in this course.

The cast (what's new this module)

- **`os.environ`** – Python's built-in dictionary of environment variables. `os.environ["DATABASE_URL"]` reads the variable; if it doesn't exist, Python raises `KeyError`. That `KeyError` is the *intended* failure mode in this module – fail loudly when configuration is missing, don't paper over it with a default.
- **`python-dotenv`** – a small library that reads a `.env` file at process start and copies any `KEY=VALUE` lines into `os.environ`. It runs **once**, at the top of `app/main.py`, before any other code reads from `os.environ`. The `.env` file is gitignored so each environment (your laptop, your teammate's, production) keeps its own values out of source control.

Together: `python-dotenv` populates `os.environ` from `.env`; the rest of your app reads `os.environ` directly. No Settings class, no global config object – just reads from the dict where the values already are. This is also why we don't use `pydantic-settings` or any “config framework”: the dict already exists.

Add python-dotenv to requirements

```
fastapi==0.115.*
uvicorn[standard]==0.32.*
jinja2==3.1.*
httpx==0.27.*
psycpg[binary]==3.2.*
python-dotenv==1.0.*
```

Create .env.example

This file is **committed** to the repo as documentation of the env contract. Each developer copies it to .env for their local machine; .env is gitignored.

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2
```

```
cp .env.example .env
```

Update app/main.py (load_dotenv at the very top, before service imports)

```
from dotenv import load_dotenv

load_dotenv() # Must run BEFORE importing modules that read os.environ at load time.

import httpx
import psycpg
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles

from app.database import get_conn
from app.schemas import AskRequest, AskResponse, Interaction
from app.services.ollama_service import OLLAMA_BASE_URL, call_ollama
from app.services.interaction_service import save_interaction, fetch_recent_history

# ... rest of main.py is unchanged from Module 7, except /healthz now uses
# OLLAMA_BASE_URL instead of the hardcoded URL ...

@app.get("/healthz")
def healthz():
    status = {"ollama": False, "postgres": False}
    try:
        with httpx.Client(timeout=5.0) as client:
            client.get(f"{OLLAMA_BASE_URL}/api/tags").raise_for_status()
            status["ollama"] = True
    except httpx.HTTPError:
        pass
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
```

```

        status["postgres"] = True
    except psycopg.Error:
        pass
    return status

```

Update app/database.py

```

import os
import psycopg
from contextlib import contextmanager

DATABASE_URL = os.environ["DATABASE_URL"]

@contextmanager
def get_conn():
    with psycopg.connect(DATABASE_URL) as conn:
        yield conn

```

Update app/services/ollama_service.py

```

import os
import httpx
from fastapi import HTTPException

OLLAMA_BASE_URL = os.environ["OLLAMA_BASE_URL"]
OLLAMA_MODEL = os.environ["OLLAMA_MODEL"]
OLLAMA_URL = f"{OLLAMA_BASE_URL}/api/chat"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)

def call_ollama(question: str) -> str:
    try:
        with httpx.Client(timeout=60.0) as client:
            r = client.post(OLLAMA_URL, json={
                "model": OLLAMA_MODEL,
                "messages": [
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": question},
                ],
                "stream": False,
            })
            r.raise_for_status()
            return r.json()["message"]["content"]
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Ollama is not reachable.")

```

Why this design

Two design choices worth understanding.

No Pydantic BaseSettings, no pydantic-settings, no Settings class of any kind. Three `os.environ["..."]` reads at module load is one line per variable. A Settings class would add an import, a class definition, an instantiation, and a `.attribute` lookup at every call site to express the same binding – more code that does the same thing. For a small app like this one, three direct reads are clearer; bring in a Settings class when you actually need its features (e.g. validation, type coercion, nested config trees).

`os.environ["X"]`, not `os.environ.get("X", default)`. A missing required variable should fail loudly at startup, not silently fall back to a hardcoded value that might be wrong in production. We *want* this to crash if `.env` isn't loaded – that crash is the operator's signal that something is misconfigured.

Also note: `SYSTEM_PROMPT` stays as a hardcoded constant in `ollama_service.py`. It's prompt *content*, not environment. The host of the database changes between dev and production. The persona of your AI assistant doesn't.

The loud-fail demo

This is the moment the loud-fail behavior becomes obvious.

Trace it in execution order

At startup, with `.env` present:

1. You run `uvicorn app.main:app`.
2. Uvicorn imports `app.main`.
3. `from dotenv import load_dotenv` → imported.
4. `load_dotenv()` → reads `.env` from the current directory, parses each line, sets `os.environ["DATABASE_URL"]`, `os.environ["OLLAMA_BASE_URL"]`, `os.environ["OLLAMA_MODEL"]`. **The current Python process now has those variables in its environment.**
5. `import httpx`, `import psycpg`, `FastAPI` imports – all proceed normally.
6. `from app.database import get_conn` → opens `database.py`, runs `DATABASE_URL = os.environ["DATABASE_URL"]` → succeeds because step 4 set it. `get_conn` is wrapped.
7. `from app.services.ollama_service import OLLAMA_BASE_URL, call_ollama` → opens `ollama_service.py`, runs `OLLAMA_BASE_URL = os.environ["OLLAMA_BASE_URL"]` and `OLLAMA_MODEL = os.environ["OLLAMA_MODEL"]` → both succeed.
8. The remaining imports and the route registrations.
9. Uvicorn starts listening. App works exactly like Module 7.

At startup, with `.env` MISSING:

1. You run `uvicorn app.main:app`.
2. Uvicorn imports `app.main`.
3. `from dotenv import load_dotenv` → imported.
4. `load_dotenv()` → tries to read `.env`, the file doesn't exist, `load_dotenv` does NOT raise (this is `load_dotenv`'s deliberate behavior – missing file is silent). **`os.environ` does NOT get the new keys.**
5. Imports continue: `import httpx`, `import psycpg`, `FastAPI` imports.
6. `from app.database import get_conn` → opens `database.py`, runs `DATABASE_URL = os.environ["DATABASE_URL"]` → **`KeyError: 'DATABASE_URL'`**. The exception propagates up. The import itself fails.
7. Uvicorn catches the import error and prints a Python traceback. **The server never starts. Port 8000 is never opened.**

That `KeyError` is the loud-fail. The operator instantly sees: missing `.env`, fix it, retry.

Notice what does NOT happen: there's no fallback to a default `DATABASE_URL`. There's no log message like "warning: using default DB". There's no scenario where the app starts but points at the wrong database.

Either it's correctly configured, or it doesn't run. That's the whole safety mechanism.

Predict before you run

Try this experiment, predicting first:

```
# Happy path with .env present
cp .env.example .env
uvicorn app.main:app --reload
# Server starts, /healthz returns both true. All good.
```

Now break it on purpose:

```
# Stop the server (Ctrl+C). Then:
mv .env .env.bak
uvicorn app.main:app
```

The server **refuses to start**. You get a traceback ending with:

```
KeyError: 'DATABASE_URL'
```

The traceback points at `app/database.py` line 5: the `os.environ["DATABASE_URL"]` line. This is *correct behaviour*. If `.env` isn't there, the operator hasn't configured the app – the app should not pretend it's fine.

Restore `.env`:

```
mv .env.bak .env
```

The server starts again.

Verify

```
./scripts/verify_module_8.sh # run from inside dist/module_08_configuration/ if testing
↪ in dist
```

Defend It

Why fail loudly on a missing env var instead of falling back to a default?

This is the most production-relevant question in the whole curriculum. Get this one right and you'll write significantly safer code for the rest of your career.

What you've built

Take a moment.

You started in Module 0 with nothing – a verified environment, no code. Eight modules later, you have:

- A FastAPI server that listens on localhost:8000 and serves an HTML page.
- A typed POST endpoint that validates input at the boundary using Pydantic.
- A backend that is *itself* a client of another local service (Ollama).
- A system-prompt-shaped LLM call that can constrain how the model responds.
- Persistent storage in Postgres – every interaction outlives the request.
- A read endpoint and UI that shows the recent ten interactions.

- A maintainable file structure with separated concerns (routes / schemas / database / services).
- Environment-driven configuration that fails loudly when misconfigured.

In summary: **nine ideas, nine working checkpoints, no module that bundled two ideas at once, every line of code you can defend.**

You can run this entire app on a laptop with no internet connection. You can explain every line to a peer. If someone asked you “what does this app do?” you could answer in one sentence. If they asked “why does it work?” you could spend an hour.

That’s the bar. That’s a serious app.

What comes next

V1 ends here. The modules below are sketched in the PRD’s Curriculum Map (Section 18) and are not implemented in this course. Each one adds exactly one fundamental on top of V1, in the same shape we’ve been using.

Module	Capability	Single fundamental
9	Chat sessions	The application provides memory the model lacks
10	File upload and document intake	Ingestion precedes retrieval
11	Keyword RAG	Retrieve, then generate with context
12	pgvector and semantic RAG	Similarity in meaning, not in words
13	Citations and grounding	Answers must be traceable to evidence
14	Streaming responses	A response can be a flow, not a packet
15	Docker Compose	Repeatable local runtimes
16	Deployment readiness	Local assumptions break in hosted environments

If you want to take a swing at any of these on your own, here’s the recipe that’s worked for V1: pick the *single* fundamental, build the *smallest* code change that makes it visible, run it, defend it. Don’t bundle two ideas. Don’t add code that doesn’t serve the fundamental. If you’re tempted to add a Settings class, a wrapper, a factory function – sit with the temptation for a minute and ask whether the lesson would survive without it. Most of the time, it does.

A good first one to try is **Module 9 (chat sessions)**. The single fundamental is “*the application provides memory the model lacks*.” The smallest change is probably: add a `session_id` column to the `interactions` table; add a `/sessions/{id}/ask` endpoint; pass the prior messages from that session into the `messages` array along with the system prompt and the new user message. Stop there. Don’t add multi-user auth. Don’t add session expiry. Don’t add a session list endpoint. One fundamental.

That’s the whole course. Nine modules, nine fundamentals, one working app. Now go build something with it.